

Port-Hamiltonian Neural ODE Networks on Lie Groups for Robot Dynamics Learning and Control

Thai Duong[✉], *Graduate Student Member, IEEE*, Abdullah Altawaitan[✉], *Student Member, IEEE*, Jason Stanley[✉], *Student Member, IEEE*, and Nikolay Atanasov[✉], *Senior Member, IEEE*

Abstract—Accurate models of robot dynamics are critical for safe and stable control and generalization to novel operational conditions. Hand-designed models, however, may be insufficiently accurate, even after careful parameter tuning. This motivates the use of machine learning techniques to approximate the robot dynamics over a training set of state-control trajectories. The dynamics of many robots are described in terms of their generalized coordinates on a matrix Lie group, e.g., on $SE(3)$ for ground, aerial, and underwater vehicles, and generalized velocity, and satisfy conservation of energy principles. This article proposes a port-Hamiltonian formulation over a Lie group of the structure of a neural ordinary differential equation (ODE) network to approximate the robot dynamics. In contrast to a black-box ODE network, our formulation embeds energy conservation principle and Lie group's constraints in the dynamics model and explicitly accounts for energy-dissipation effect such as friction and drag forces in the dynamics model. We develop energy shaping and damping injection control for the learned, potentially under-actuated Hamiltonian dynamics to enable a unified approach for stabilization and trajectory tracking with various robot platforms.

Index Terms— $SE(3)$ manifold, dynamics learning, Hamiltonian dynamics, neural ordinary differential equation (ODE) networks.

I. INTRODUCTION

MOTION planning and optimal control algorithms depend on the availability of accurate system dynamics models. Models obtained from first principles and calibrated over a small set of parameters via system identification [1] are widely used for unmanned ground vehicles (UGVs), unmanned aerial

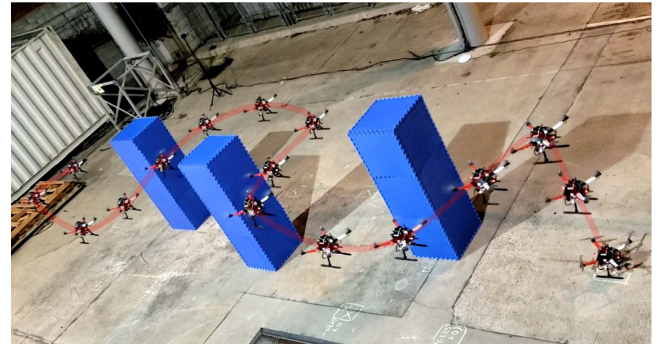


Fig. 1. Quadrotor trajectory tracking using a learned port-Hamiltonian dynamics model.

vehicles (UAVs), and unmanned underwater vehicles (UUVs). Such models may oversimplify or even incorrectly describe the underlying structure of the dynamical system, leading to bias and modeling errors that cannot be corrected by adjusting a few parameters. Data-driven techniques [2], [3], [4], [5], [6] have emerged as a powerful approach to approximate system dynamics with an overparameterized machine learning model, trained over a dataset of system state and control trajectories. Neural networks are expressive function approximation models, capable of identifying and generalizing interaction patterns from the training data. Training neural network models, however, typically requires large amounts of data and computation time, which may be impractical in mobile robotics applications. Recent works [7], [8], [9], [10], [11], [12], [13], [14] have considered a hybrid (gray-box) approach, where prior knowledge of the physics, governing the system dynamics, is used to assist the learning process. The dynamics of physical systems obey kinematic constraints and energy conservation laws. These laws are known to be universally true but a black-box machine learning model might struggle to infer them from the training data, causing poor generalization. Instead, prior knowledge may be encoded into the learning model, e.g., using a prior distribution [3], a graph-network forward kinematic model [15], or a network architecture reflecting the structure of Lagrangian [16] or Hamiltonian [10] mechanical systems. Moreover, many physical robot platforms are composed of rigid-body interconnections and their state evolution respects the structure of a Lie group [17], e.g., the position and orientation kinematics of a rigid body evolve on the $SE(3)$ Lie group [18]. Existing works [19], [20], [21] on Lie group neural ordinary differential equations (ODEs)

Manuscript received 11 January 2024; revised 7 June 2024; accepted 18 June 2024. Date of publication 15 July 2024; date of current version 7 August 2024. This work was supported by NSF CCF-2112665 (TILOS) and NSF FRR CAREER 2045945. This article was recommended for publication by Associate Editor C. Della Santina and Editor J. Bohg upon evaluation of the reviewers' comments. (Corresponding author: Thai Duong.)

Thai Duong, Jason Stanley, and Nikolay Atanasov are with the Department of Electrical and Computer Engineering, University of California San Diego, La Jolla, CA 92093 USA (e-mail: tduong@ucsd.edu; jstanle@ucsd.edu; natanasov@ucsd.edu).

Abdullah Altawaitan is with the Department of Electrical and Computer Engineering, University of California San Diego, La Jolla, CA 92093 USA, and also with Kuwait University 13060, Kuwait (e-mail: aaltawaitan@ucsd.edu).

This video illustrates the approach to solve the problem of learning robot dynamics, whose states evolve on a matrix Lie group, from a training set of state-control trajectories and the problem of tracking a trajectory with the learned dynamics. The video also shows the proposed method working in simulation and real-world demonstrations with quadrotors. Software and videos are available at <https://thaipduong.github.io/LieGroupHamDL>.

This article has supplementary downloadable material available at <https://doi.org/10.1109/TRO.2024.3428433>, provided by the authors.

Digital Object Identifier 10.1109/TRO.2024.3428433

networks for learning dynamics and normalizing flows focus on preservation of the Lie group structure during backpropagation using an adjoint method, either via a higher dimensional space [19], [20] or local coordinates [21], [22]. The goal of this article is to incorporate both the kinematic structure and the energy conservation properties of physical systems with Lie group states into the structure of a dynamics learning model. We also aim to design a control approach that achieves stabilization or trajectory tracking using the learned model without requiring prior knowledge of its parameters. In other words, the same control design should enable trajectory tracking for learned models of different rigid-body UGVs, UAVs, or UUVs.

Lagrangian and Hamiltonian mechanics [23], [24] provide physical system descriptions that can be integrated into the structure of a neural network [10], [11], [25], [26], [27], [28]. Prior work, however, has only considered vector-valued states, when designing Lagrangian- or Hamiltonian-structured neural networks. This limits the applicability of these techniques as many common robot systems have states on a Lie group. For example, Hamiltonian equations of motion are available for orientation but existing formulations rely predominantly on 3-D vector parametrizations, such as Euler angles [29], [30], which suffer from singularities. Similar to [19] and [20], our work embeds matrix Lie groups in a higher dimensional space $\mathbb{R}^{n \times n}$, allowing us to train the model via the widely used neural ODE network on Euclidean space [31]. However, we focus on embedding the Hamiltonian formulation of robot dynamics in the model and deriving a control policy for trajectory tracking. Concurrently, Wotte et al. [22] offer an exciting approach to learn SE(3) Hamiltonian dynamics from data using a neural ODE on Lie group, trained via local coordinates [21] with guarantees of satisfaction of Lie group constraints. The authors designed an adjoint method on the Lie algebra for neural ODE training, and a potential shaping controller for stabilization of a fully actuated rigid body.

Our preliminary work [32] designed a neural ODE network [31] to capture Hamiltonian dynamics on the SE(3) manifold [33], and derived a trajectory tracking control policy for potentially underactuated systems. Our model is shown to provide accurate long-term trajectory predictions, respecting SE(3) constraints and conserve total energy with high precision. Inspired by [27] and [34], we model kinetic energy and potential energy by separate neural networks, each governed by a set of Hamiltonian equations on SE(3). However, our preliminary work [32] is developed specifically for the SE(3) manifold and does not model dissipation elements that drain energy from the system, such as friction or drag forces in real robot systems. The Hamiltonian formulation of robot dynamics can be generalized to port-Hamiltonian formulation, where the dynamics are governed by energy exchange, i.e., the law of energy conservation, energy dissipation, e.g., from friction, and energy injection, e.g., from control inputs. In this article, we generalize our dynamics learning and control method in [32] using a port-Hamiltonian neural ODE network to embed general matrix Lie group constraints and introduce an energy dissipation term, represented by another neural network, to model friction and air drag in physical systems. We compensate for energy dissipation

in the trajectory tracking control design to provide accurate tracking performance. We verify our approach extensively with simulated robot systems, including a pendulum and a Crazyflie quadrotor, and with several real quadrotor platforms (see Fig. 1). In summary, this article makes the following *contributions*.

- 1) We design a neural ODE model that respects port-Hamiltonian dynamics over a *matrix Lie group* to enable data-driven learning of robot dynamics.
- 2) We develop a unified control policy for port-Hamiltonian dynamics on a Lie group that achieves trajectory tracking if permissible by the system's degree of underactuation.
- 3) We demonstrate our dynamics learning and control approach extensively with simulated robot systems (a pendulum and a quadrotor) and several real quadrotor robots.

II. RELATED WORK

Data-driven techniques [35], [36], [37], [38], [39] have shown impressive results in learning robot dynamics models from state-control trajectories. Neural networks offer especially expressive system models but their training requires large amounts of data, which may be impractical in mobile robot applications. Recently, a hybrid approach [7], [8], [9], [10], [11], [12], [13], [14], [28], [40], [41], [42] has been considered where prior knowledge about a physical system is integrated into the design of a machine learning model. Models designed with structure respecting kinematic constraints [15], symmetry [43], [44], Lagrangian mechanics [7], [8], [9], [12], [13], [16], [45], or Hamiltonian mechanics [10], [11], [14], [25], [26], [27], [28], [46], [47] guarantee that the laws of physics are satisfied by construction, regardless of the training data.

Sanchez-Gonzalez et al. [15] design graph neural networks to represent the kinematic structure of complex dynamical systems and demonstrate forward model learning and online planning via gradient-based trajectory optimization. Ruthotto et al. [43] propose a partial differential equation (PDE) interpretation of convolutional neural networks and derive new parabolic and hyperbolic ResNet architectures guided by PDE theory. Wang et al. [44] design symmetry equivariant neural network models, encoding rotation, scaling, and uniform motion, to learn physical dynamics that are robust to symmetry group distributional shifts.

Lagrangian-based methods [7], [8], [9], [12], [13], [16], [45] design neural network models for physical systems based on the Euler-Lagrange differential equations of motion [23], [24], in terms of generalized coordinates $\mathbf{q}$, their velocity $\dot{\mathbf{q}}$ and a Lagrangian function $\mathcal{L}(\mathbf{q}, \dot{\mathbf{q}})$, defined as the difference between the kinetic and potential energies. The energy terms are modeled by neural networks, either separately [7], [16], [45] or together [9].

Hamiltonian-based methods [10], [11], [14], [25], [26], [27], [28] use a Hamiltonian formulation [23], [24] of the system dynamics, instead, in terms of generalized coordinates $\mathbf{q}$, generalized momenta $\mathbf{p}$, and a Hamiltonian function, $\mathcal{H}(\mathbf{q}, \mathbf{p})$, representing the total energy of the system. Greydanus et al. [10] model the Hamiltonian as a neural network and update its parameters by minimizing the discrepancy between its symplectic gradients and the time derivatives of the states $(\mathbf{q}, \mathbf{p})$. This approach,

however, requires that the state time derivatives are available in the training data set. Chen et al. [11] and Zhong et al. [27] relax this assumption by using differentiable leapfrog integrators [48] and differentiable ODE solvers [31], respectively. The need for time derivatives of the states is eliminated by back-propagating a loss function measuring state discrepancy through the ODE solvers via the adjoint method. Our work extends the approach in [27], [34] by formulating the Hamiltonian dynamics over a matrix Lie group, which enforces kinematic constraints in the neural ODE network used to learn the dynamics. Toth et al. [49] and Mason et al. [50] show that, instead of from state trajectories, the Hamiltonian function can be learned from high-dimensional image observations. Finzi et al. [26] show that using Cartesian coordinates with explicit constraints improves both the accuracy and data efficiency for the Lagrangian- and Hamiltonian-based approaches. In a closely related work, Zhong et al. [34] showed that dissipating elements, such as friction or air drag, can be incorporated in a Hamiltonian-based neural ODE network by reformulating the system dynamics in port-Hamiltonian form [51]. The continuous-time equations of motions in Lagrangian or Hamiltonian dynamics can also be discretized using variational integrators [52] to learn discrete-time Lagrangian and Hamiltonian systems [53], [54], [55], [56] and provide long-term prediction for control methods such as model predictive control [57]. This approach eliminates the need to use an ODE solver to roll out the dynamics but its prediction accuracy depends on the discretization time step. Meanwhile, our work encodes not only the Hamiltonian structure but also the Lie group constraints, satisfied by the states of rigid-body robot systems, such as UGVs, UAVs, and UUVs, in a neural ODE network to learn robot dynamics from data.

While most existing dynamics learning approaches focus on Euclidean dynamics, many robot systems have states evolving on a matrix Lie group. Recent works on neural ODE networks [19], [20], [21], [22] on Lie groups are classified into either extrinsic [19], [20] or intrinsic methods [21], [22]. The extrinsic approach [19], [20] embeds a Lie group in a higher dimensional space with Lie group constraints, enabling training with neural ODEs on Euclidean space. Our work belongs to the extrinsic approach by enforcing matrix Lie group constraints on its embedding space $\mathbb{R}^{n \times n}$, but focuses on incorporating the law of energy conservation, via a Hamiltonian formulation, in the dynamics model and providing control design for trajectory tracking. Meanwhile, the intrinsic approach [21], [22] develops adjoint methods for training using local coordinates on the Lie algebra, e.g., via local charts [21], and therefore, guarantees Lie groups constraints by design. Concurrently to our work, Wotte et al. [22] develop a neural ODE network on Lie group to learn Hamiltonian dynamics on the SE(3) manifold by deriving an adjoint method on the Lie algebra, offering an exciting approach for learning structure-preserving dynamics model.

While few dynamics learning articles consider control design based on the learned model, we develop a general trajectory tracking controller for Lie group Hamiltonian dynamics. The Hamiltonian formulation and its port-Hamiltonian generalization [51] are built around the notion of system

energy, and hence, are naturally related to control techniques for stabilization aiming to minimize the total energy. Since the minimum point of the Hamiltonian might not correspond to a desired regulation point, the control design needs to inject additional energy to ensure that the minimum of the total energy is at the desired equilibrium. For fully actuated port-Hamiltonian systems, it is sufficient to shape the potential energy only using an energy-shaping and damping-injection controller [51]. For underactuated systems, both the kinetic and potential energies need to be shaped, e.g., via interconnection and damping assignment passivity-based control (IDA-PBC) [51], [58], [59], [60]. Wang and Goldsmith [61] extend the IDA-PBC controller from stabilization to trajectory tracking. Closely related to our controller design, Souza et al. [62] apply this technique to design a controller for an underactuated quadrotor robot but use Euler angles as the orientation representation. Port-Hamiltonian structure and energy-based control design are also used to learn distributed control policies from state-control trajectories [63], [64], [65].

We connect Hamiltonian-dynamics learning with the idea of IDA-PBC control to allow stabilization of any rigid-body robot without relying on its model parameters a priori. We design a trajectory-tracking controller for underactuated systems, e.g., quadrotor robots, based on the IDA-PBC approach and show how to construct desired pose and momentum trajectories given only desired position and yaw. We demonstrate the tight integration of dynamics learning and control to achieve closed-loop trajectory tracking with underactuated quadrotor robots.

III. PROBLEM STATEMENT

Consider a robot with state $\mathbf{x}$ consisting of generalized coordinates $\mathbf{q}$ evolving on a Lie group G and generalized velocity ξ on the Lie algebra $\mathfrak{g}$ of G . Let $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$ characterize the robot dynamics with control input $\mathbf{u} \in \mathbb{R}^m$. For example, the state of rigid-body mobile robot, such as a UGV or UAV, may be modeled by its pose on the SE(3) group, consisting of position and orientation, and its twist on the $\mathfrak{se}(3)$ Lie algebra, consisting of linear and angular velocity. The control input of an Ackermann-drive UGV may include its linear acceleration and steering angle rate, and that of a quadrotor UAV may include the total thrust and moment generated by the propellers; see Section IV-C for more details.

We assume that the function $\mathbf{f}$ specifying the robot dynamics is unknown and aim to approximate it using a dataset $\mathcal{D}$ of state and control trajectories. Specifically, let $\mathcal{D} = \{t_{0:N}^{(i)}, \mathbf{x}_{0:N}^{(i)}, \mathbf{u}^{(i)}\}_{i=1}^D$ consist of D state sequences $\mathbf{x}_{0:N}^{(i)}$, obtained by applying a constant control input $\mathbf{u}^{(i)}$ to the system with initial condition $\mathbf{x}_0^{(i)}$ at time $t_0^{(i)}$ and sampling its state $\mathbf{x}^{(i)}(t_n^{(i)}) =: \mathbf{x}_n^{(i)}$ at times $t_0^{(i)} < t_1^{(i)} < \dots < t_N^{(i)}$. Using the dataset $\mathcal{D}$, we aim to find a function $\bar{\mathbf{f}}_\theta$ with parameters θ that approximates the true dynamics $\mathbf{f}$ well. To optimize θ , we roll out the approximate dynamics $\bar{\mathbf{f}}_\theta$ with initial state $\mathbf{x}_0^{(i)}$ and constant control $\mathbf{u}^{(i)}$ and minimize the discrepancy between the computed state sequence $\bar{\mathbf{x}}_{1:N}^{(i)}$ and the true state sequence $\mathbf{x}_{1:N}^{(i)}$ in $\mathcal{D}$.

Problem 1: Given a dataset $\mathcal{D} = \{t_{0:N}^{(i)}, \mathbf{x}_{0:N}^{(i)}, \mathbf{u}^{(i)}\}_{i=1}^D$ and a function $\bar{\mathbf{f}}_\theta$, find the parameters θ that minimize

$$\begin{aligned} \min_{\theta} \quad & \sum_{i=1}^D \sum_{n=1}^N \ell(\mathbf{x}_n^{(i)}, \bar{\mathbf{x}}_n^{(i)}) \\ \text{s.t.} \quad & \dot{\bar{\mathbf{x}}}^{(i)}(t) = \bar{\mathbf{f}}_\theta(\bar{\mathbf{x}}^{(i)}(t), \mathbf{u}^{(i)}), \quad \bar{\mathbf{x}}^{(i)}(t_0) = \mathbf{x}_0^{(i)} \\ & \bar{\mathbf{x}}_n^{(i)} = \bar{\mathbf{x}}^{(i)}(t_n) \quad \forall n = 1, \dots, N \quad \forall i = 1, \dots, D \end{aligned} \quad (1)$$

where ℓ is a distance metric on the state space.

Furthermore, we aim to design a feedback controller capable of tracking a desired state trajectory $\mathbf{x}^*(t), t \geq t_0$, for the learned model $\bar{\mathbf{f}}_\theta$ of the robot dynamics.

Problem 2: Given an initial condition $\mathbf{x}_0$ at time t_0 , desired state trajectory $\mathbf{x}^*(t), t \geq t_0$, and learned dynamics $\bar{\mathbf{f}}_\theta$, design a feedback control law $\mathbf{u} = \pi(\mathbf{x}, \theta, \mathbf{x}^*(t))$ such that $\limsup_{t \rightarrow \infty} \ell(\mathbf{x}(t), \mathbf{x}^*(t))$ is bounded.

We consider robot kinematics on the Lie group G such that when there is no control input, $\mathbf{u} = \mathbf{0}$, the dynamics $\mathbf{f}(\mathbf{x}, \mathbf{u})$ respect the law of energy conservation. We embed these constraints in the structure of the parametric function $\bar{\mathbf{f}}_\theta$. We review matrix Lie groups, with the $SE(3)$ manifold as an example, and Hamiltonian dynamics equations next.

IV. PRELIMINARIES

A. Matrix Lie Groups

In this section, we cover the background needed to define Hamiltonian dynamics on a Lie group. Refer to [17], [33], and [66] for a more detailed overview of matrix Lie groups.

Definition 1 (Dot Product): The dot product $\langle \cdot, \cdot \rangle$ between two matrices ξ and ψ in $\mathbb{R}^{n \times m}$ can be chosen as

$$\langle \xi, \psi \rangle = \text{tr}(\xi^\top \psi). \quad (2)$$

The aforementioned dot product definition is used to define the dual maps in Definitions 6 and 9, and loss functions in Sections V-C and V-D.

Definition 2 (General linear group [17]): The general linear group $GL(n, \mathbb{R})$ is the group of $n \times n$ invertible real matrices.

Definition 3 (Matrix Lie group [17]): A matrix Lie group G is a subgroup of $GL(n, \mathbb{R})$ with identity element $\mathbf{e}$ such that if any sequence of matrices $\{A_n\}_{n=0}^\infty$ in G converges to a matrix A , then either A is in G or A is not invertible. A matrix Lie group is also a smooth embedded submanifold on $\mathbb{R}^{n \times n}$.

Definition 4 (Tangent space and bundle): The tangent space $T_{\mathbf{q}}G$ is the set of all tangent vectors ξ to the manifold G at $\mathbf{q}$. The tangent bundle TG is the set of all the pairs $(\mathbf{q}, \xi)$ with $\mathbf{q} \in G$ and $\xi \in T_{\mathbf{q}}G$.

Definition 5 (Lie algebra and Lie bracket): A Lie algebra is a vector space $\mathfrak{g}$, equipped with a Lie bracket operator $[\cdot, \cdot] : \mathfrak{g} \times \mathfrak{g} \rightarrow \mathfrak{g}$ that satisfies

$$\text{bilinearity: } [a\xi_1 + b\xi_2, \xi_3] = a[\xi_1, \xi_3] + b[\xi_2, \xi_3]$$

$$[\xi_3, a\xi_1 + b\xi_2] = a[\xi_3, \xi_1] + b[\xi_3, \xi_2]$$

$$\text{skew-symmetry: } [\xi_1, \xi_2] = -[\xi_2, \xi_1]$$

$$\text{Jacobi identity:}$$

$$[\xi_1, [\xi_2, \xi_3]] + [\xi_2, [\xi_3, \xi_1]] + [\xi_3, [\xi_1, \xi_2]] = 0.$$

Every matrix Lie group G is associated with a Lie algebra $\mathfrak{g}$, which is the tangent space at the identity element $T_{\mathbf{e}}G$. An element $\mathbf{q} \in G$ is linked with an element $\xi \in \mathfrak{g}$ via the exponential map $\exp_G : \mathfrak{g} \rightarrow G$ and the logarithm map $\log_G : G \rightarrow \mathfrak{g}$ [17]. Since tangent spaces of G , and in particular, the Lie algebra $\mathfrak{g}$, are isomorphic to Euclidean space, we can define a linear mapping $(\cdot)^\wedge : \mathbb{R}^n \rightarrow \mathfrak{g}$ and its inverse $(\cdot)^\vee : \mathfrak{g} \rightarrow \mathbb{R}^n$, where n is the dimension of G . Thus, we can map between G and $\mathbb{R}^n$ using the compositions

$$\exp_G^\wedge = \exp_G \circ \wedge, \quad \log_G^\vee = \vee \circ \log_G. \quad (3)$$

In this article, we consider a matrix Lie group element $\mathbf{q} \in G$ embedded in $\mathbb{R}^{n \times n}$, instead of its n -dimensional representations, e.g., $\log_G^\vee \mathbf{q}$, due to potential issues of discontinuity [67] and singularity [68], [69]. For example, Zhou et al. [67] show that higher dimensional representations of rotations, e.g., $(n^2 - n)$ dimensions for $SO(n)$, are more suitable for learning using neural networks because they ensure continuity.

Definition 6 (Left translation and invariant vectors): The left translation $L_{\mathbf{q}} : G \rightarrow G$ with $\mathbf{q} \in G$ is defined as

$$L_{\mathbf{q}}(\mathbf{h}) = \mathbf{q}\mathbf{h}. \quad (4)$$

The left-invariant vector $T_{\mathbf{e}}L_{\mathbf{q}}(\xi)$ is defined as the derivative of the left translation $L_{\mathbf{q}}$ at $\mathbf{h} = \mathbf{e}$ in the direction of ξ . This vector describes the kinematics of the Lie group, which relates the velocity $\xi \in \mathfrak{g}$ to the change $\dot{\mathbf{q}} \in T_{\mathbf{q}}G$ of coordinates $\mathbf{q}$

$$\dot{\mathbf{q}} = T_{\mathbf{e}}L_{\mathbf{q}}(\xi) = \mathbf{q}\xi. \quad (5)$$

Given a pairing $\langle \cdot, \cdot \rangle$ on $\mathfrak{g}^* \times \mathfrak{g}$ (e.g., Definition 1), the dual map $T_{\mathbf{e}}^*L_{\mathbf{q}}$ of $T_{\mathbf{e}}L_{\mathbf{q}}$ satisfies

$$\langle T_{\mathbf{e}}^*L_{\mathbf{q}}(\eta), \xi \rangle = \langle \eta, T_{\mathbf{e}}L_{\mathbf{q}}(\xi) \rangle \quad (6)$$

for any $\eta \in \mathfrak{g}^*$ and $\xi \in \mathfrak{g}$.

Definition 7 (Adjoint Operator): For $\mathbf{q} \in G$, the adjoint $\text{Ad}_{\mathbf{q}} : \mathfrak{g} \rightarrow \mathfrak{g}$ is defined as

$$\text{Ad}_{\mathbf{q}}(\psi) = \mathbf{q}\psi\mathbf{q}^{-1}. \quad (7)$$

The algebra adjoint $\text{ad}_{\xi} : \mathfrak{g} \rightarrow \mathfrak{g}$ is the directional derivative of $\text{Ad}_{\mathbf{q}}$ at $\mathbf{q} = \mathbf{e}$ in the direction of $\xi \in \mathfrak{g}$

$$\text{ad}_{\xi}(\psi) = \left. \frac{d}{dt} \text{Ad}_{\exp_G(t\xi)}(\psi) \right|_{t=0} = [\xi, \psi]. \quad (8)$$

Definition 8 (Cotangent space and bundle): The dual space of the tangent space $T_{\mathbf{q}}G$, i.e., the space of all linear functionals from $T_{\mathbf{q}}G$ to $\mathbb{R}$, is called the cotangent space $T_{\mathbf{q}}^*G$. At the identity $\mathbf{e}$, the cotangent space of the Lie algebra $\mathfrak{g} = T_{\mathbf{e}}G$ is denoted $\mathfrak{g}^*$. The cotangent bundle T^*G is the set of all the pairs $(\mathbf{q}, \mathbf{p})$ with $\mathbf{q} \in G$ and $\mathbf{p} \in T_{\mathbf{q}}^*G$.

Definition 9 (Coadjoint operator): For $\mathbf{q} \in G$, the coadjoint $\text{Ad}_{\mathbf{q}}^* : \mathfrak{g}^* \rightarrow \mathfrak{g}^*$ is defined as $\langle \text{Ad}_{\mathbf{q}}^*(\varphi), \psi \rangle = \langle \varphi, \text{Ad}_{\mathbf{q}}(\psi) \rangle$, where $\varphi \in \mathfrak{g}^*$, $\psi \in \mathfrak{g}$ and $\langle \cdot, \cdot \rangle$ is a pairing on $\mathfrak{g}^* \times \mathfrak{g}$. The algebra coadjoint $\text{ad}_{\xi}^* : \mathfrak{g}^* \rightarrow \mathfrak{g}^*$ is the dual map of ad_{ξ} , satisfying $\langle \text{ad}_{\xi}^*(\varphi), \psi \rangle = \langle \varphi, \text{ad}_{\xi}(\psi) \rangle$.

The next section describes the $SE(3)$ Lie group to illustrate the aforementioned definitions. The $SE(3)$ Lie group is used to represent the position and orientation of a rigid body.

B. Example: $SE(3)$ Manifold

Consider a fixed world inertial frame of reference and a rigid body with a body-fixed frame attached to its center of mass. The pose of the body-fixed frame in the world frame is determined by the position $\mathbf{p} = [x, y, z]^\top \in \mathbb{R}^3$ of the center of mass and the orientation of the body-fixed frame's coordinate axes

$$\mathbf{R} = [\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{r}_3]^\top \in SO(3) \quad (9)$$

where $\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3 \in \mathbb{R}^3$ are the rows of the rotation matrix $\mathbf{R}$. A rotation matrix is an element of the special orthogonal group

$$SO(3) = \{\mathbf{R} \in \mathbb{R}^{3 \times 3} : \mathbf{R}^\top \mathbf{R} = \mathbf{I}, \det(\mathbf{R}) = 1\}. \quad (10)$$

The rigid-body position and orientation can be combined in a single pose matrix $\mathbf{q} \in SE(3)$, which is an element of the special Euclidean group

$$SE(3) = \left\{ \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}^\top & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 4} : \mathbf{R} \in SO(3), \mathbf{p} \in \mathbb{R}^3 \right\}. \quad (11)$$

The kinematic equations of motion of the rigid body are determined by the linear velocity $\mathbf{v} \in \mathbb{R}^3$ and angular velocity $\boldsymbol{\omega} \in \mathbb{R}^3$ of the body-fixed frame with respect to the world frame, expressed in body-frame coordinates. The generalized velocity $\boldsymbol{\zeta} = [\mathbf{v}^\top, \boldsymbol{\omega}^\top]^\top \in \mathbb{R}^6$ determines the rate of change of the rigid-body pose according to the $SE(3)$ kinematics

$$\dot{\mathbf{q}} = \mathbf{q}\boldsymbol{\xi} = \mathbf{q}\hat{\boldsymbol{\zeta}} =: \mathbf{q} \begin{bmatrix} \hat{\boldsymbol{\omega}} & \mathbf{v} \\ \mathbf{0}^\top & 0 \end{bmatrix} \quad (12)$$

where we overload $\hat{\cdot}$ to denote the mapping from a vector $\boldsymbol{\zeta} \in \mathbb{R}^6$ to a 4×4 twist matrix $\boldsymbol{\xi} = \hat{\boldsymbol{\zeta}}$ in the Lie algebra $\mathfrak{se}(3)$ of $SE(3)$ and from a vector $\boldsymbol{\omega} \in \mathbb{R}^3$ to a 3×3 skew-symmetric matrix $\hat{\boldsymbol{\omega}}$ in the Lie algebra $\mathfrak{so}(3)$ of $SO(3)$

$$\hat{\boldsymbol{\omega}} = \begin{bmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{bmatrix}. \quad (13)$$

Refer to [68] for an excellent introduction to the use of $SE(3)$ in robot state estimation problems.

C. Hamiltonian Dynamics on Matrix Lie Groups

In this section, we describe Hamilton's equations of motion on a matrix Lie group [33], [66]. Our neural network architecture design in Section V is based on Lie group Hamiltonian dynamics, which encode both kinematic constraints and energy conservation.

Consider a system with generalized coordinates $\mathbf{q}$ in a matrix Lie group G and generalized velocity $\dot{\mathbf{q}} \in T_{\mathbf{q}}G$. The dynamics of the state $\mathbf{x} = (\mathbf{q}, \dot{\mathbf{q}}) \in TG$ satisfy

$$\dot{\mathbf{q}} = T_{\mathbf{e}}L_{\mathbf{q}}(\boldsymbol{\xi}) = \mathbf{q}\boldsymbol{\xi} \quad (14)$$

where $\boldsymbol{\xi}$ is an element in the Lie algebra $\mathfrak{g}$. The Lagrangian on a Lie group $\mathcal{L} : G \times \mathfrak{g} \rightarrow \mathbb{R}$ is defined as the difference between the kinetic energy $\mathcal{T} : G \times \mathfrak{g} \rightarrow \mathbb{R}$ and the potential energy $\mathcal{V} : G \rightarrow \mathbb{R}$

$$\mathcal{L}(\mathbf{q}, \boldsymbol{\xi}) = \mathcal{T}(\mathbf{q}, \boldsymbol{\xi}) - \mathcal{V}(\mathbf{q}). \quad (15)$$

The Hamiltonian is obtained using a Legendre transformation

$$\mathcal{H}(\mathbf{q}, \mathbf{p}) = \mathbf{p} \cdot \boldsymbol{\xi} - \mathcal{L}(\mathbf{q}, \boldsymbol{\xi}) \quad (16)$$

where the momentum $\mathbf{p}$ is defined as

$$\mathbf{p} = \frac{\partial \mathcal{L}(\mathbf{q}, \boldsymbol{\xi})}{\partial \boldsymbol{\xi}}. \quad (17)$$

The state $(\mathbf{q}, \mathbf{p}) \in T^*G$ evolves according to the Hamiltonian dynamics [33] as

$$\dot{\mathbf{q}} = T_{\mathbf{e}}L_{\mathbf{q}} \left(\frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}} \right) \quad (18a)$$

$$\dot{\mathbf{p}} = \text{ad}_{\boldsymbol{\xi}}^*(\mathbf{p}) - T_{\mathbf{e}}^*L_{\mathbf{q}} \left(\frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{q}} \right) + \mathbf{B}(\mathbf{q})\mathbf{u}. \quad (18b)$$

Obtaining explicit expressions for $\text{ad}_{\boldsymbol{\xi}}^*(\mathbf{p})$ and $T_{\mathbf{e}}^*L_{\mathbf{q}}(\boldsymbol{\eta})$ with $\mathbf{q} \in G$, $\boldsymbol{\xi} \in \mathfrak{g}$, and $\mathbf{p}, \boldsymbol{\eta} \in \mathfrak{g}^*$ depends on the structure of the matrix Lie algebra $\mathfrak{g}$ and the pairing $\langle \cdot, \cdot \rangle$ on $\mathfrak{g}^* \times \mathfrak{g}$. Appendix B provides details and an example for the $SE(3)$ manifold.

By comparing (14) and (18), we have

$$\boldsymbol{\xi} = \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}}. \quad (19)$$

Let $\boldsymbol{\eta} = \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{q}}$. When there is no control input, i.e., $\mathbf{u} = 0$, the conservation of energy is guaranteed as

$$\begin{aligned} \frac{d\mathcal{H}(\mathbf{q}, \mathbf{p})}{dt} &= \langle \boldsymbol{\eta}, \dot{\mathbf{q}} \rangle + \langle \boldsymbol{\xi}, \dot{\mathbf{p}} \rangle \\ &= \langle \boldsymbol{\eta}, T_{\mathbf{e}}L_{\mathbf{q}}(\boldsymbol{\xi}) \rangle - \langle \boldsymbol{\xi}, T_{\mathbf{e}}^*L_{\mathbf{q}}(\boldsymbol{\eta}) \rangle + \langle \boldsymbol{\xi}, \text{ad}_{\boldsymbol{\xi}}^*(\mathbf{p}) \rangle \\ &= 0 \end{aligned} \quad (20)$$

because of (6), and by definition,

$$\langle \boldsymbol{\xi}, \text{ad}_{\boldsymbol{\xi}}^*(\mathbf{p}) \rangle = \langle \text{ad}_{\boldsymbol{\xi}}(\boldsymbol{\xi}), \mathbf{p} \rangle = \langle [\boldsymbol{\xi}, \boldsymbol{\xi}], \mathbf{p} \rangle = 0. \quad (21)$$

D. Reformulation as Port-Hamiltonian Dynamics

The notion of energy in dynamical systems is shared across multiple domains, including mechanical, electrical, and thermal. A port-Hamiltonian generalization [51] of Hamiltonian mechanics is used to model systems with energy-storing elements (e.g., kinetic and potential energy), energy-dissipating elements (e.g., friction or resistance), and external energy sources (e.g., control inputs), connected via energy ports. An input-state-output port-Hamiltonian system has the form

$$\begin{bmatrix} \dot{\mathbf{q}} \\ \dot{\mathbf{p}} \end{bmatrix} = (\mathcal{J}(\mathbf{q}, \mathbf{p}) - \mathcal{R}(\mathbf{q}, \mathbf{p})) \begin{bmatrix} \frac{\partial \mathcal{H}}{\partial \mathbf{q}} \\ \frac{\partial \mathcal{H}}{\partial \mathbf{p}} \end{bmatrix} + \mathcal{G}(\mathbf{q}, \mathbf{p})\mathbf{u} \quad (22)$$

where $\mathcal{J}(\mathbf{q}, \mathbf{p})$ is a skew-symmetric interconnection matrix, representing the energy-storing elements, $\mathcal{R}(\mathbf{q}, \mathbf{p}) \succeq 0$ is a positive semidefinite dissipation matrix, representing the energy-dissipating elements, and $\mathcal{G}(\mathbf{q}, \mathbf{p})$ is an input matrix such that

$\mathcal{G}(\mathbf{q}, \mathbf{p})\mathbf{u}$ represents the external energy sources. In the absence of energy-dissipating elements and external energy sources, the skew-symmetry of $\mathcal{J}(\mathbf{q}, \mathbf{p})$ guarantees the energy conservation of the system.

To model energy dissipating elements such as friction or drag forces, we reformulate the Hamiltonian dynamics on a matrix Lie group (18) in port-Hamiltonian form (22). Such elements are often modeled [70] as a linear transformation $\mathbf{D}(\mathbf{q}, \mathbf{p}) \succeq 0$ of the velocity $\dot{\boldsymbol{\xi}}$ and only affect the generalized momenta $\mathbf{p}$, i.e.,

$$\mathcal{R}(\mathbf{q}, \mathbf{p}) = \begin{bmatrix} \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{D}(\mathbf{q}, \mathbf{p}) \end{bmatrix}. \quad (23)$$

The Hamiltonian dynamics (18) is a special case of (22), where the dissipation matrix is $\mathbf{D}(\mathbf{q}, \mathbf{p}) = \mathbf{0}$, the input matrix is $\mathcal{G}(\mathbf{q}, \mathbf{p}) = [\mathbf{0}^\top \quad \mathbf{B}(\mathbf{q})^\top]^\top$ and the interconnection matrix $\mathcal{J}(\mathbf{q}, \mathbf{p})$ can be obtained by rearranging (18) with $\mathbf{u} = 0$ and is guaranteed to be skew-symmetric due to the energy conservation (20). In an implementation, the coordinates $\mathbf{q}$ and momentum $\mathbf{p}$ may be represented as vectors in $\mathbb{R}^{n^2}$, leading to an interconnection matrix $\mathcal{J}(\mathbf{q}, \mathbf{p}) \in \mathbb{R}^{2n^2 \times 2n^2}$.

E. Example: Hamiltonian Dynamics on the SE(3) Manifold

In this section, we consider the generalized coordinate $\mathbf{q}$ of a mobile robot consisting of its position $\mathbf{p} \in \mathbb{R}^3$ and orientation $\mathbf{R} \in SO(3)$. Let $\mathbf{q} = (\mathbf{p}, \mathbf{R})$ be the generalized coordinates and $\boldsymbol{\zeta} = (\mathbf{v}, \boldsymbol{\omega}) \in \mathbb{R}^6$ be the generalized velocity, consisting of the body-frame linear velocity $\mathbf{v} \in \mathbb{R}^3$ and the body-frame angular velocity $\boldsymbol{\omega} \in \mathbb{R}^3$. The coordinate $\mathbf{q}$ evolves on the Lie group $SE(3)$ while the generalized velocity satisfies $\dot{\mathbf{q}} = \mathbf{q}\boldsymbol{\xi}$, where $\boldsymbol{\xi} = \hat{\boldsymbol{\zeta}}$ is a twist matrix in $\mathfrak{se}(3)$, as shown in (12). The isomorphism between $\mathfrak{se}(3)$ and $\mathbb{R}^6$ via (12) simplifies the Hamiltonian (18) and its port-Hamiltonian formulation (22) as follows. The Lagrangian function on $SE(3)$ can be expressed in terms of $\mathbf{q}$ and $\boldsymbol{\zeta}$, instead of $\mathbf{q}$ and $\dot{\mathbf{q}}$ as follows:

$$\mathcal{L}(\mathbf{q}, \boldsymbol{\zeta}) = \frac{1}{2} \boldsymbol{\zeta}^\top \mathbf{M}(\mathbf{q}) \boldsymbol{\zeta} - \mathcal{V}(\mathbf{q}). \quad (24)$$

The generalized mass matrix has a block-diagonal form when the body frame is attached to the center of mass [33]

$$\mathbf{M}(\mathbf{q}) = \begin{bmatrix} \mathbf{M}_v(\mathbf{q}) & \mathbf{0} \\ \mathbf{0} & \mathbf{M}_\omega(\mathbf{q}) \end{bmatrix} \in \mathbb{S}_{>0}^{6 \times 6} \quad (25)$$

where $\mathbf{M}_v(\mathbf{q}), \mathbf{M}_\omega(\mathbf{q}) \in \mathbb{S}_{>0}^{3 \times 3}$. The generalized momenta are defined, as before, via the partial derivative of the Lagrangian with respect to the twist

$$\mathbf{p} = \begin{bmatrix} \mathbf{p}_v \\ \mathbf{p}_\omega \end{bmatrix} = \frac{\partial \mathcal{L}(\mathbf{q}, \boldsymbol{\zeta})}{\partial \boldsymbol{\zeta}} = \mathbf{M}(\mathbf{q}) \boldsymbol{\zeta} \in \mathbb{R}^6. \quad (26)$$

The Hamiltonian function of the system becomes

$$\mathcal{H}(\mathbf{q}, \mathbf{p}) = \mathbf{p} \cdot \boldsymbol{\zeta} - \mathcal{L}(\mathbf{q}, \boldsymbol{\zeta}) = \frac{1}{2} \mathbf{p}^\top \mathbf{M}^{-1}(\mathbf{q}) \mathbf{p} + \mathcal{V}(\mathbf{q}). \quad (27)$$

By vectorizing the generalized coordinates $\mathbf{q} = [\mathbf{p}^\top \quad \mathbf{r}_1^\top \quad \mathbf{r}_2^\top \quad \mathbf{r}_3^\top]^\top$, the Hamiltonian dynamics on $SE(3)$ can be described

in port-Hamiltonian form (22) [33], [71], [72] with interconnection matrix

$$\mathcal{J}(\mathbf{q}, \mathbf{p}) = \begin{bmatrix} \mathbf{0} & \mathbf{q}^\times \\ -\mathbf{q}^{\times\top} & \mathbf{p}^\times \end{bmatrix}, \quad \mathbf{p}^\times = \begin{bmatrix} \mathbf{0} & \hat{\mathbf{p}}_v \\ \hat{\mathbf{p}}_v & \hat{\mathbf{p}}_\omega \end{bmatrix} \quad (28)$$

and input matrix $\mathcal{G}(\mathbf{q}, \mathbf{p}) = [\mathbf{0}^\top \quad \mathbf{B}(\mathbf{q})^\top]^\top$, where $\mathbf{q}^\times = \begin{bmatrix} \mathbf{R}^\top & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \hat{\mathbf{r}}_1^\top & \hat{\mathbf{r}}_2^\top & \hat{\mathbf{r}}_3^\top \end{bmatrix}^\top$. Note that the kinematics constraints $\dot{\mathbf{q}} = \mathbf{q}\boldsymbol{\xi}$ of the coordinates $\mathbf{q}$ on $SE(3)$, which guarantee that the coordinates $\mathbf{q}$ remain in $SE(3)$, are not affected by vectorization. The port-Hamiltonian formulation allows us to model dissipation elements in the dynamics by the dissipation matrix

$$\mathbf{D}(\mathbf{q}, \mathbf{p}) = \begin{bmatrix} \mathbf{D}_v(\mathbf{q}, \mathbf{p}) & \mathbf{0} \\ \mathbf{0} & \mathbf{D}_\omega(\mathbf{q}, \mathbf{p}) \end{bmatrix} \in \mathbb{S}_{>0}^{6 \times 6} \quad (29)$$

where the components $\mathbf{D}_v(\mathbf{q}, \mathbf{p})$ and $\mathbf{D}_\omega(\mathbf{q}, \mathbf{p})$ correspond to $\mathbf{p}_v$ and $\mathbf{p}_\omega$, respectively. The equations of motions on the $SE(3)$ manifold are written in port-Hamiltonian form as

$$\dot{\mathbf{p}} = \mathbf{R} \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_v} \quad (30a)$$

$$\dot{\mathbf{r}}_i = \mathbf{r}_i \times \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_\omega}, \quad i = 1, 2, 3 \quad (30b)$$

$$\begin{aligned} \dot{\mathbf{p}}_v &= \mathbf{p}_v \times \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_\omega} - \mathbf{R}^\top \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}} \\ &\quad - \mathbf{D}_v(\mathbf{q}, \mathbf{p}) \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_v} + \mathbf{b}_v(\mathbf{q})\mathbf{u} \end{aligned} \quad (30c)$$

$$\begin{aligned} \dot{\mathbf{p}}_\omega &= \mathbf{p}_\omega \times \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_\omega} + \mathbf{p}_v \times \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_v} \\ &\quad + \sum_{i=1}^3 \mathbf{r}_i \times \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{r}_i} - \mathbf{D}_\omega(\mathbf{q}, \mathbf{p}) \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_\omega} + \mathbf{b}_\omega(\mathbf{q})\mathbf{u} \end{aligned} \quad (30d)$$

where the input matrix is $\mathbf{B}(\mathbf{q}) = [\mathbf{b}_v(\mathbf{q})^\top \quad \mathbf{b}_\omega(\mathbf{q})^\top]^\top$.

F. Neural ODE Networks

In this section, we briefly describe neural ODE networks [31], which approximate the closed-loop dynamics $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \boldsymbol{\pi}(\mathbf{x}))$ of a system for some unknown control policy $\mathbf{u} = \boldsymbol{\pi}(\mathbf{x})$ by a neural network $\bar{\mathbf{f}}_\theta(\mathbf{x})$. The parameters of $\bar{\mathbf{f}}_\theta(\mathbf{x})$ are trained using a dataset $\mathcal{D} = \{t_{0:N}^{(i)}, \mathbf{x}_{0:N}^{(i)}\}_i$ of state trajectory samples $\mathbf{x}_n^{(i)} = \mathbf{x}^{(i)}(t_n^{(i)})$ via forward and backward passes through a differentiable ODE solver, where the backward passes provide the gradient of the loss function. Given an initial state $\mathbf{x}_0^{(i)}$ at time $t_0^{(i)}$, a forward pass returns predicted states at times $t_1^{(i)}, \dots, t_N^{(i)}$ as follows:

$$\{\bar{\mathbf{x}}_1^{(i)}, \dots, \bar{\mathbf{x}}_N^{(i)}\} = \text{ODESolver}(\mathbf{x}_0^{(i)}, \bar{\mathbf{f}}_\theta, t_1^{(i)}, \dots, t_N^{(i)}). \quad (31)$$

The gradient of a loss function, $\sum_{i=1}^D \sum_{j=1}^N \ell(\mathbf{x}_j^{(i)}, \bar{\mathbf{x}}_j^{(i)})$, is back-propagated by solving another ODE with adjoint states. The parameters θ are updated by gradient descent to minimize

the loss. For physical systems, Zhong et al. [27] extends the neural ODE by integrating the Hamiltonian dynamics on $\mathbb{R}^n$ into the neural network model $\bar{\mathbf{f}}_\theta(\mathbf{x})$, and consider zero-order hold control input $\mathbf{u}$, leading to a neural ODE network with the following approximated dynamics:

$$\begin{bmatrix} \dot{\mathbf{x}} \\ \dot{\mathbf{u}} \end{bmatrix} = \begin{bmatrix} \bar{\mathbf{f}}_\theta(\mathbf{x}, \mathbf{u}) \\ \mathbf{0} \end{bmatrix}. \quad (32)$$

Recently, neural ODE networks have been extended from Euclidean space to Lie groups [19], [20], [21], [22], guaranteeing that the Lie group constraints are satisfied by the predicted states by design. While it is possible to train our Hamiltonian dynamics model using a Lie group neural ODE network, we leave this investigation for future work due to the lack of suitable open-source software for Lie group ODE integration.

V. LEARNING LIE GROUP HAMILTONIAN DYNAMICS

We consider a Hamiltonian system with unknown kinetic energy $\mathcal{T}(\mathbf{q})$, potential energy $\mathcal{V}(\mathbf{q})$, input matrix $\mathbf{B}(\mathbf{q})$, dissipation matrix $\mathbf{D}(\mathbf{q}, \mathbf{p})$, and design a structured neural ODE network to learn these terms from state-control trajectories.

A. Data Collection

We collect a dataset $\mathcal{D} = \{\mathbf{x}_{0:N}^{(i)}, \mathbf{x}_{0:N}^{(i)}, \mathbf{u}^{(i)}\}_{i=1}^D$ consisting of state sequences $\mathbf{x}_{0:N}^{(i)}$, where $\mathbf{x}_n^{(i)} = [\mathbf{q}_n^{(i)\top} \boldsymbol{\xi}_n^{(i)\top}]^\top$ for $n = 0, \dots, N$. Such data are generated by applying a constant control input $\mathbf{u}^{(i)}$ to the system and sampling the state $\mathbf{x}_n^{(i)} = \mathbf{x}^{(i)}(t_n^{(i)})$ at times $t_n^{(i)}$ for $n = 0, \dots, N$. The generalized coordinates $\mathbf{q}$ and velocity $\boldsymbol{\xi}$ may be obtained from a state estimation algorithm, such as odometry algorithm for mobile robots [73], [74], or from a motion capture system. In physics-based simulation, the data can be generated by applying random control inputs $\mathbf{u}^{(i)}$. In real-world applications, where safety is a concern, data may be collected by a human operator manually controlling the robot.

B. Model Architecture

Since robots are physical systems, their dynamics $\mathbf{f}(\mathbf{x}, \mathbf{u})$ satisfy the Hamiltonian formulation (see Section IV-C). To learn the dynamics $\mathbf{f}(\mathbf{x}, \mathbf{u})$ from a trajectory dataset $\mathcal{D}$, we design a neural ODE network (see Section IV-F), approximating the dynamics via a parametric function $\bar{\mathbf{f}}_\theta(\mathbf{x}, \mathbf{u})$ based on (18).

To integrate the Hamiltonian equations into the structure of $\bar{\mathbf{f}}_\theta(\mathbf{x}, \mathbf{u})$, we use four neural networks with parameters $\theta = (\theta_\mathcal{T}, \theta_\mathcal{V}, \theta_\mathbf{D}, \theta_\mathbf{B})$ to approximate the kinetic energy by $\mathcal{T}_\theta(\mathbf{q}, \boldsymbol{\xi})$, the potential energy by $\mathcal{V}_\theta(\mathbf{q})$, the dissipation matrix by $\mathbf{D}_\theta(\mathbf{q}, \mathbf{p})$, and the input matrix by $\mathbf{B}_\theta(\mathbf{q})$, respectively. Since the generalized momenta $\mathbf{p}$ are not directly available in $\mathcal{D}$, the time derivative of the generalized velocity $\boldsymbol{\xi}$ is obtained from (19). The approximated dynamics function $\bar{\mathbf{f}}_\theta(\mathbf{x}, \mathbf{u})$ is described with an internal state $\mathbf{p}$ as follows:

$$\dot{\mathbf{q}} = \mathbf{T}_e \mathbf{L}_q \left(\frac{\partial \mathcal{H}_\theta(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}} \right) \quad (33a)$$

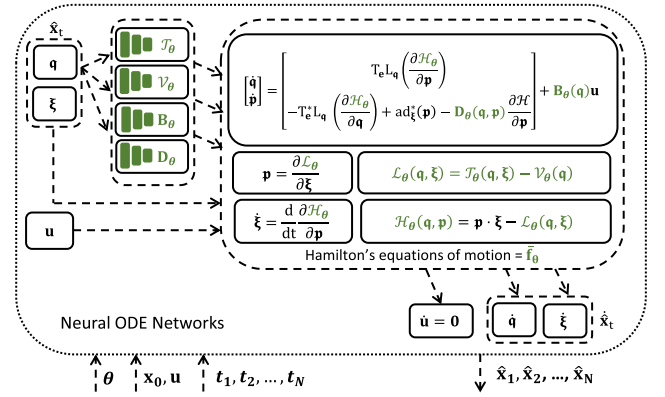


Fig. 2. Architecture of port-Hamiltonian neural ODE network on matrix Lie group. The trainable terms are shown in green.

$$\begin{aligned} \dot{\mathbf{p}} = & \mathbf{ad}_\xi^*(\mathbf{p}) - \mathbf{D}_\theta(\mathbf{q}, \mathbf{p}) \frac{\partial \mathcal{H}_\theta(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}} \\ & - \mathbf{T}_e^* \mathbf{L}_q \left(\frac{\partial \mathcal{H}_\theta(\mathbf{q}, \mathbf{p})}{\partial \mathbf{q}} \right) + \mathbf{B}_\theta(\mathbf{q}) \mathbf{u} \end{aligned} \quad (33b)$$

$$\dot{\boldsymbol{\xi}} = \frac{d}{dt} \frac{\partial \mathcal{H}_\theta(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}} \quad (33c)$$

where $\mathcal{H}_\theta(\mathbf{q}, \mathbf{p}) = \mathbf{p} \cdot \boldsymbol{\xi} - \mathcal{L}_\theta(\mathbf{q}, \boldsymbol{\xi})$, and $\mathcal{L}_\theta(\mathbf{q}, \boldsymbol{\xi}) = \mathcal{T}_\theta(\mathbf{q}, \boldsymbol{\xi}) - \mathcal{V}_\theta(\mathbf{q})$. The time derivative $\frac{d}{dt} \frac{\partial \mathcal{H}_\theta(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}}$ is calculated using automatic differentiation, e.g., by Pytorch [75]. The approximated dynamics function $\bar{\mathbf{f}}_\theta(\mathbf{x}, \mathbf{u})$ is implemented in a neural ODE network architecture for training, shown in Fig. 2.

C. Training Process

Let $\bar{\mathbf{x}}^{(i)}(t)$ denote the state trajectory predicted with control input $\mathbf{u}^{(i)}$ by the approximate dynamics $\bar{\mathbf{f}}_\theta$ initialized at $\bar{\mathbf{x}}^{(i)}(t_0^{(i)}) = \mathbf{x}_0^{(i)}$. For sequence i , forward passes through the ODE solver in (31) return the predicted states $\bar{\mathbf{x}}_{0:N}^{(i)}$ at times $t_{0:N}^{(i)}$, where $\bar{\mathbf{x}}_n^{(i)} = [\bar{\mathbf{q}}_n^{(i)\top} \bar{\boldsymbol{\xi}}_n^{(i)\top}]^\top$, for $n = 1, \dots, N$. The predicted coordinates $\bar{\mathbf{q}}_n^{(i)}$ and the ground-truth ones $\mathbf{q}_n^{(i)}$ are used to calculate a loss on the Lie group manifold

$$L_q(\theta) = \sum_{i=1}^D \sum_{n=1}^N \left\| \log_{\mathbf{q}_n^{(i)}} \left(\bar{\mathbf{q}}_n^{(i)} \left(\mathbf{q}_n^{(i)} \right)^{-1} \right) \right\|_2^2. \quad (34)$$

We use the squared Euclidean norm to calculate losses for the generalized velocity terms

$$L_\xi(\theta) = \sum_{i=1}^D \sum_{n=1}^N \left\| \boldsymbol{\xi}_n^{(i)} - \bar{\boldsymbol{\xi}}_n^{(i)} \right\|_2^2. \quad (35)$$

The total loss $L(\theta)$ is defined as

$$L(\theta) = L_q(\theta) + L_\xi(\theta). \quad (36)$$

The gradient of the total loss function $L(\theta)$ is back-propagated by solving an ODE with adjoint states [31]. Specifically, let $\mathbf{a} = \frac{\partial L}{\partial \bar{\mathbf{x}}}$ be the adjoint state and $\mathbf{s} = (\bar{\mathbf{x}}, \mathbf{a}, \frac{\partial L}{\partial \theta})$ be the augmented

state. The augmented state dynamics are [31]

$$\dot{\mathbf{s}} = \bar{\mathbf{f}}_{\mathbf{s}} = \left(\bar{\mathbf{f}}_{\boldsymbol{\theta}}, -\mathbf{a}^\top \frac{\partial \bar{\mathbf{f}}_{\boldsymbol{\theta}}}{\partial \bar{\mathbf{x}}}, -\mathbf{a}^\top \frac{\partial \bar{\mathbf{f}}_{\boldsymbol{\theta}}}{\partial \boldsymbol{\theta}} \right). \quad (37)$$

The predicted state $\bar{\mathbf{x}}$, the adjoint state $\mathbf{a}$, and the derivatives $\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}}$ can be obtained by a single call to a reverse-time ODE solver starting from $\mathbf{s}_N = \mathbf{s}(t_N)$

$$\mathbf{s}_0 = \left(\bar{\mathbf{x}}_0, \mathbf{a}_0, \frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}} \right) = \text{ODESolver}(\mathbf{s}_N, \bar{\mathbf{f}}_{\mathbf{s}}, t_N) \quad (38)$$

where at each time $t_k, k = 1, \dots, N$, the adjoint state $\mathbf{a}_k$ at time t_k is reset to $\frac{\partial \mathcal{L}}{\partial \bar{\mathbf{x}}_k}$. The resulting derivative $\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}}$ is used to update the parameters $\boldsymbol{\theta}$ using gradient descent. Note that even though the Lie group and Hamiltonian structures are preserved in the continuous-time dynamics function $\bar{\mathbf{f}}$, the Lie group constraints might still be violated by the predicted states when the model is trained with a neural ODE network on the embedding space $\mathbb{R}^{n \times n}$ instead of on the matrix Lie group [21], [22]. To prevent this, we use a high-order integrator, such as fifth-order Runge–Kutta [76], in the Euclidean space neural ODE, for which open-source software is available [31], [77]. Investigating how to train Hamiltonian models with a neural ODE network defined directly on the Lie group [21], [22], [78] is an interesting future direction. For example, the Lie group neural ODE by Wotte et al. [22] offers an approach to guarantee Lie group constraints in the dynamics model. Closely related to this direction, Duruisseaux et al. [54] learn discrete-time Hamiltonian dynamics while guaranteeing Lie group constraints by design using variational integration.

D. Application to SE(3) Hamiltonian Dynamics Learning

This section applies our Lie group Hamiltonian dynamics learning approach to estimate mobile robot dynamics on the SE(3) manifold (see Section IV-E).

1) *Neural ODE Model Architecture*: When the Hamiltonian dynamics in (18) are defined on the SE(3) manifold, the equations of motion become (30). The neural ODE network architecture in (33) is simplified as follows. We use five neural networks with parameters $\boldsymbol{\theta} = (\boldsymbol{\theta}_{\mathbf{v}}, \boldsymbol{\theta}_{\boldsymbol{\omega}}, \boldsymbol{\theta}_V, \boldsymbol{\theta}_{\mathbf{D}}, \boldsymbol{\theta}_{\mathbf{B}})$ to approximate the blocks $\mathbf{M}_{\mathbf{v};\boldsymbol{\theta}}^{-1}(\mathbf{q})$, $\mathbf{M}_{\boldsymbol{\omega};\boldsymbol{\theta}}^{-1}(\mathbf{q})$ of the inverse generalized mass in (25), the potential energy $\mathcal{V}_{\boldsymbol{\theta}}(\mathbf{q})$, the dissipation matrix $\mathbf{D}_{\boldsymbol{\theta}}(\mathbf{q}, \mathbf{p})$, and the input matrix $\mathbf{B}_{\boldsymbol{\theta}}(\mathbf{q})$, respectively. The approximated kinetic energy is calculated as $\mathcal{T}_{\boldsymbol{\theta}}(\mathbf{q}, \mathbf{p}) = \frac{1}{2} \mathbf{p}^\top \mathbf{M}_{\boldsymbol{\theta}}^{-1}(\mathbf{q}) \mathbf{p}$, where $\mathbf{M}_{\boldsymbol{\theta}}(\mathbf{q}) = \text{diag}(\mathbf{M}_{\mathbf{v};\boldsymbol{\theta}}(\mathbf{q}), \mathbf{M}_{\boldsymbol{\omega};\boldsymbol{\theta}}(\mathbf{q}))$.

2) *Neural Network Design*: In many applications, nominal information is available about the generalized mass matrices $\mathbf{M}_{\mathbf{v};\boldsymbol{\theta}}^{-1}(\mathbf{q})$, $\mathbf{M}_{\boldsymbol{\omega};\boldsymbol{\theta}}^{-1}(\mathbf{q})$, the potential energy $\mathcal{V}_{\boldsymbol{\theta}}(\mathbf{q})$, the dissipation matrix $\mathbf{D}_{\boldsymbol{\theta}}(\mathbf{q}, \mathbf{p})$, and the input matrix $\mathbf{B}_{\boldsymbol{\theta}}(\mathbf{q})$, and can be included in the neural network design.

Let $\mathbf{M}_{\mathbf{v}0}^{-1}(\mathbf{q})$, $\mathbf{M}_{\boldsymbol{\omega}0}^{-1}(\mathbf{q})$, and $\mathbf{D}_0(\mathbf{q}, \mathbf{p})$ be the nominal values of the generalized mass matrices $\mathbf{M}_{\mathbf{v};\boldsymbol{\theta}}^{-1}(\mathbf{q})$, $\mathbf{M}_{\boldsymbol{\omega};\boldsymbol{\theta}}^{-1}(\mathbf{q})$ and the dissipation matrix $\mathbf{D}_{\boldsymbol{\theta}}(\mathbf{q}, \mathbf{p})$ with Cholesky decomposition

$$\begin{aligned} \mathbf{M}_{\mathbf{v}0}^{-1}(\mathbf{q}) &= \mathbf{L}_{\mathbf{v}0}(\mathbf{q}) \mathbf{L}_{\mathbf{v}0}^\top(\mathbf{q}) \\ \mathbf{M}_{\boldsymbol{\omega}0}^{-1}(\mathbf{q}) &= \mathbf{L}_{\boldsymbol{\omega}0}(\mathbf{q}) \mathbf{L}_{\boldsymbol{\omega}0}^\top(\mathbf{q}) \\ \mathbf{D}_0(\mathbf{q}) &= \mathbf{L}_{\mathbf{D}0}(\mathbf{q}) \mathbf{L}_{\mathbf{D}0}^\top(\mathbf{q}). \end{aligned} \quad (39)$$

The learned terms $\mathbf{M}_{\mathbf{v};\boldsymbol{\theta}}^{-1}(\mathbf{q})$, $\mathbf{M}_{\boldsymbol{\omega};\boldsymbol{\theta}}^{-1}(\mathbf{q})$, and $\mathbf{D}_{\boldsymbol{\theta}}(\mathbf{q}, \mathbf{p})$ are obtained using Cholesky decomposition

$$\begin{aligned} \mathbf{M}_{\mathbf{v};\boldsymbol{\theta}}^{-1}(\mathbf{q}) &= (\mathbf{L}_{\mathbf{v}0}(\mathbf{q}) + \mathbf{L}_{\mathbf{v}}(\mathbf{q})) (\mathbf{L}_{\mathbf{v}0}(\mathbf{q}) + \mathbf{L}_{\mathbf{v}}(\mathbf{q}))^\top + \varepsilon_{\mathbf{v}} \mathbf{I} \\ \mathbf{M}_{\boldsymbol{\omega};\boldsymbol{\theta}}^{-1}(\mathbf{q}) &= (\mathbf{L}_{\boldsymbol{\omega}0}(\mathbf{q}) + \mathbf{L}_{\boldsymbol{\omega}}(\mathbf{q})) (\mathbf{L}_{\boldsymbol{\omega}0}(\mathbf{q}) + \mathbf{L}_{\boldsymbol{\omega}}(\mathbf{q}))^\top + \varepsilon_{\boldsymbol{\omega}} \mathbf{I} \\ \mathbf{D}_{\boldsymbol{\theta}}(\mathbf{q}, \mathbf{p}) &= (\mathbf{L}_{\mathbf{D}0}(\mathbf{q}, \mathbf{p}) + \mathbf{L}_{\mathbf{D}}(\mathbf{q}, \mathbf{p})) (\mathbf{L}_{\mathbf{D}0}(\mathbf{q}, \mathbf{p}) + \mathbf{L}_{\mathbf{D}}(\mathbf{q}, \mathbf{p}))^\top \end{aligned} \quad (40)$$

where $\mathbf{L}_{\mathbf{v}}(\mathbf{q})$, $\mathbf{L}_{\boldsymbol{\omega}}(\mathbf{q})$, and $\mathbf{L}_{\mathbf{D}}(\mathbf{q}, \mathbf{p})$ are lower triangular matrices implemented as three neural networks with parameters $\boldsymbol{\theta}_{\mathbf{v}}$, $\boldsymbol{\theta}_{\boldsymbol{\omega}}$, and $\boldsymbol{\theta}_{\mathbf{D}}$, respectively, and $\varepsilon_{\mathbf{v}}, \varepsilon_{\boldsymbol{\omega}} > 0$.

The potential energy $\mathcal{V}(\mathbf{q})$ and the input matrix $\mathbf{B}(\mathbf{q})$ are implemented with nominal values $\mathcal{V}_0(\mathbf{q})$ and $\mathbf{B}_0(\mathbf{q})$ as follows:

$$\begin{aligned} \mathcal{V}_{\boldsymbol{\theta}}(\mathbf{q}) &= \mathcal{V}_0(\mathbf{q}) + \mathcal{V}_{\mathbf{v}}(\mathbf{q}) \\ \mathbf{B}_{\boldsymbol{\theta}}(\mathbf{q}) &= \mathbf{B}_0(\mathbf{q}) + \mathbf{L}_{\mathbf{B}}(\mathbf{q}) \end{aligned} \quad (41)$$

where $\mathcal{V}_{\mathbf{v}}(\mathbf{q})$ and $\mathbf{L}_{\mathbf{B}}(\mathbf{q})$ are two neural networks with parameters $\boldsymbol{\theta}_{\mathbf{v}}$ and $\boldsymbol{\theta}_{\mathbf{B}}$, respectively.

3) *Loss Function*: The orientation loss is calculated as

$$L_{\mathbf{R}}(\boldsymbol{\theta}) = \sum_{i=1}^D \sum_{n=1}^N \left\| \log_{SO(3)}^{\mathbf{v}}(\bar{\mathbf{R}}_n^{(i)} \mathbf{R}_n^{(i)\top}) \right\|_2^2. \quad (42)$$

We use the squared Euclidean norm to calculate losses for the position and generalized velocity terms

$$\begin{aligned} L_{\mathbf{p}}(\boldsymbol{\theta}) &= \sum_{i=1}^D \sum_{n=1}^N \left\| \mathbf{p}_n^{(i)} - \bar{\mathbf{p}}_n^{(i)} \right\|_2^2 \\ L_{\boldsymbol{\zeta}}(\boldsymbol{\theta}) &= \sum_{i=1}^D \sum_{n=1}^N \left\| \boldsymbol{\zeta}_n^{(i)} - \bar{\boldsymbol{\zeta}}_n^{(i)} \right\|_2^2. \end{aligned} \quad (43)$$

The total loss $\mathcal{L}(\boldsymbol{\theta})$ is defined as

$$L(\boldsymbol{\theta}) = L_{\mathbf{R}}(\boldsymbol{\theta}) + L_{\mathbf{p}}(\boldsymbol{\theta}) + L_{\boldsymbol{\zeta}}(\boldsymbol{\theta}). \quad (44)$$

VI. ENERGY-BASED CONTROL DESIGN

The function $\bar{\mathbf{f}}_{\boldsymbol{\theta}}$ (33) learned in Section V satisfies the port-Hamiltonian dynamics in (22) by design. This section extends the interconnection and damping assignment passivity-based control (IDA-PBC) approach [51], [61], [62] to Lie groups to achieve trajectory tracking (Problem 2) based on the learned port-Hamiltonian dynamics. We further derive a tracking controller specifically for learned Hamiltonian dynamics on the SE(3) manifold in Section V-D. In the remainder of this article, we omit the subscript $\boldsymbol{\theta}$ in the learned model for readability.

A. IDA-PBC Control Design for Trajectory Tracking

Consider a desired regulation point $(\mathbf{q}^*, \mathbf{p}^*) \in \mathcal{T}^*\mathcal{G}$ that the system should be stabilized to. The Hamiltonian function $\mathcal{H}(\mathbf{q}, \mathbf{p})$, representing the total energy of the system, generally does not have a minimum at $(\mathbf{q}^*, \mathbf{p}^*)$. An IDA-PBC controller [51], [58], [61] is designed to inject additional energy $\mathcal{H}_a(\mathbf{q}, \mathbf{p})$ such that the desired total energy

$$\mathcal{H}_d(\mathbf{q}, \mathbf{p}) = \mathcal{H}(\mathbf{q}, \mathbf{p}) + \mathcal{H}_a(\mathbf{q}, \mathbf{p}) \quad (45)$$

achieves its minimum at $(\mathbf{q}^*, \mathbf{p}^*)$. In other words, the closed-loop system obtained by applying the controller to the port-Hamiltonian dynamics in (22) should have the form

$$\begin{bmatrix} \dot{\mathbf{q}} \\ \dot{\mathbf{p}} \end{bmatrix} = (\mathcal{J}_d(\mathbf{q}, \mathbf{p}) - \mathcal{R}_d(\mathbf{q}, \mathbf{p})) \begin{bmatrix} \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}} \\ \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}} \end{bmatrix} \quad (46)$$

to ensure that $(\mathbf{q}^*, \mathbf{p}^*)$ is an equilibrium. The control input $\mathbf{u}$ should be chosen so that (22) and (46) are equal. This matching equation design does not directly apply to trajectory tracking, especially for underactuated systems [61], [62].

Consider a desired trajectory $(\mathbf{q}^*(t), \mathbf{p}^*(t))$ that the system should track. Let $(\mathbf{q}_e(t), \mathbf{p}_e(t))$ denote the error in the generalized coordinates and momentum, respectively, where $\mathbf{q}_e = (\mathbf{q}^*)^{-1} \mathbf{q} \in \mathbf{G}$ and $\mathbf{p}_e = \mathbf{p} - \mathbf{p}^* \in \mathbf{T}_{\mathbf{q}_e}^* \mathbf{G}$. For trajectory tracking, the desired total energy $\mathcal{H}_d(\mathbf{q}_e, \mathbf{p}_e)$ is defined in terms of the error state, with desired closed-loop dynamics

$$\begin{bmatrix} \dot{\mathbf{q}}_e \\ \dot{\mathbf{p}}_e \end{bmatrix} = (\mathcal{J}_d(\mathbf{q}_e, \mathbf{p}_e) - \mathcal{R}_d(\mathbf{q}_e, \mathbf{p}_e)) \begin{bmatrix} \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} \\ \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \end{bmatrix}. \quad (47)$$

Matching (47) with (22) leads to the following requirement for the control input:

$$\begin{aligned} \mathcal{G}(\mathbf{q}, \mathbf{p})\mathbf{u} &= (\mathcal{J}_d(\mathbf{q}_e, \mathbf{p}_e) - \mathcal{R}_d(\mathbf{q}_e, \mathbf{p}_e)) \begin{bmatrix} \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} \\ \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \end{bmatrix} \\ &\quad - (\mathcal{J}(\mathbf{q}, \mathbf{p}) - \mathcal{R}(\mathbf{q}, \mathbf{p})) \begin{bmatrix} \frac{\partial \mathcal{H}}{\partial \mathbf{q}} \\ \frac{\partial \mathcal{H}}{\partial \mathbf{p}} \end{bmatrix} + \begin{bmatrix} \dot{\mathbf{q}} \\ \dot{\mathbf{p}} \end{bmatrix} - \begin{bmatrix} \dot{\mathbf{q}}_e \\ \dot{\mathbf{p}}_e \end{bmatrix}. \end{aligned} \quad (48)$$

The control input can be obtained from (48) as the sum $\mathbf{u} = \mathbf{u}_{ES} + \mathbf{u}_{DI}$ of an energy-shaping component $\mathbf{u}_{ES}$ and a damping-injection component $\mathbf{u}_{DI}$

$$\mathbf{u}_{ES} = \mathcal{G}^\dagger(\mathbf{q}, \mathbf{p}) \left((\mathcal{J}_d(\mathbf{q}_e, \mathbf{p}_e)) \begin{bmatrix} \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} \\ \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \end{bmatrix} - \begin{bmatrix} \dot{\mathbf{q}}_e \\ \dot{\mathbf{p}}_e \end{bmatrix} \right) \quad (49a)$$

$$- (\mathcal{J}(\mathbf{q}, \mathbf{p}) - \mathcal{R}(\mathbf{q}, \mathbf{p})) \begin{bmatrix} \frac{\partial \mathcal{H}}{\partial \mathbf{q}} \\ \frac{\partial \mathcal{H}}{\partial \mathbf{p}} \end{bmatrix} + \begin{bmatrix} \dot{\mathbf{q}} \\ \dot{\mathbf{p}} \end{bmatrix} \Bigg)$$

$$\mathbf{u}_{DI} = -\mathcal{G}^\dagger(\mathbf{q}, \mathbf{p}) \mathcal{R}_d(\mathbf{q}_e, \mathbf{p}_e) \begin{bmatrix} \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} \\ \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \end{bmatrix} \quad (49b)$$

where $\mathcal{G}^\dagger(\mathbf{q}, \mathbf{p}) = (\mathcal{G}^\top(\mathbf{q}, \mathbf{p})\mathcal{G}(\mathbf{q}, \mathbf{p}))^{-1}\mathcal{G}^\top(\mathbf{q}, \mathbf{p})$ is the pseudoinverse of $\mathcal{G}(\mathbf{q}, \mathbf{p})$. The control input $\mathbf{u}_{ES}$ exists as long as the desired interconnection matrix $\mathcal{J}_d$, dissipation matrix $\mathcal{R}_d$, and total energy $\mathcal{H}_d$ satisfy the following matching condition for all $(\mathbf{q}, \mathbf{p}) \in \mathbf{T}^*\mathbf{G}$ and $(\mathbf{q}_e, \mathbf{p}_e) \in \mathbf{T}^*\mathbf{G}$:

$$\begin{aligned} \mathcal{G}^\dagger(\mathbf{q}, \mathbf{p}) \left((\mathcal{J}_d(\mathbf{q}_e, \mathbf{p}_e) - \mathcal{R}_d(\mathbf{q}_e, \mathbf{p}_e)) \begin{bmatrix} \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} \\ \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \end{bmatrix} \right. \\ \left. - (\mathcal{J}(\mathbf{q}, \mathbf{p}) - \mathcal{R}(\mathbf{q}, \mathbf{p})) \begin{bmatrix} \frac{\partial \mathcal{H}}{\partial \mathbf{q}} \\ \frac{\partial \mathcal{H}}{\partial \mathbf{p}} \end{bmatrix} + \begin{bmatrix} \dot{\mathbf{q}} \\ \dot{\mathbf{p}} \end{bmatrix} - \begin{bmatrix} \dot{\mathbf{q}}_e \\ \dot{\mathbf{p}}_e \end{bmatrix} \right) = 0 \end{aligned} \quad (50)$$

where $\mathcal{G}^\perp(\mathbf{q}, \mathbf{p})$ is a maximal-rank left annihilator of $\mathcal{G}(\mathbf{q}, \mathbf{p})$, i.e., $\mathcal{G}^\perp(\mathbf{q}, \mathbf{p})\mathcal{G}(\mathbf{q}, \mathbf{p}) = \mathbf{0}$.

B. Tracking Control for Port-Hamiltonian Dynamics on the $SE(3)$ Manifold

Consider a desired state trajectory $\mathbf{x}^*(t) = (\mathbf{q}^*(t), \zeta^*(t))$ that the system should track where $\mathbf{q}^*(t) \in SE(3)$ is the desired pose and $\zeta^*(t) = [\mathbf{v}^*(t)^\top \ \boldsymbol{\omega}^*(t)^\top]^\top$ is the desired generalized velocity expressed in the desired frame. Let $\mathbf{p}^* = \mathbf{M} \begin{bmatrix} \mathbf{R}^\top \mathbf{R}^* \mathbf{v}^* \\ \mathbf{R}^\top \mathbf{R}^* \boldsymbol{\omega}^* \end{bmatrix}$

denote the desired momentum, defined based on (26) with the desired velocity expressed in the body frame. Let $\mathbf{p}_e = \mathbf{p} - \mathbf{p}^*$ and $\mathbf{R}_e = \mathbf{R}^{*\top} \mathbf{R} = [\mathbf{r}_{e1} \ \mathbf{r}_{e2} \ \mathbf{r}_{e3}]^\top$ be the position error and rotation error between the current orientation $\mathbf{R}$ and the desired one $\mathbf{R}^*$, respectively. The vectorized error $\mathbf{q}_e$ in the generalized coordinates is

$$\mathbf{q}_e = [\mathbf{p}_e^\top \ \mathbf{r}_{e1}^\top \ \mathbf{r}_{e2}^\top \ \mathbf{r}_{e3}^\top]^\top. \quad (51)$$

The error in the generalized momenta is $\mathbf{p}_e = \mathbf{p} - \mathbf{p}^*$, described in the body frame. The desired total energy is defined in terms of the error state as

$$\mathcal{H}_d(\mathbf{q}_e, \mathbf{p}_e) = \frac{1}{2} \mathbf{p}_e^\top \mathbf{M}_d^{-1}(\mathbf{q}_e) \mathbf{p}_e + V_d(\mathbf{q}_e) \quad (52)$$

where $\mathbf{M}_d(\mathbf{q}_e)$ and $V_d(\mathbf{q}_e)$ are the desired generalized mass and potential energy. Choosing the following desired interconnection matrix and dissipation matrix:

$$\mathcal{J}_d(\mathbf{q}_e, \mathbf{p}_e) = \begin{bmatrix} \mathbf{0} & \mathbf{J}_1 \\ -\mathbf{J}_1^\top & \mathbf{J}_2 \end{bmatrix}, \quad \mathcal{R}_d(\mathbf{q}_e, \mathbf{p}_e) = \begin{bmatrix} \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{K}_d \end{bmatrix} \quad (53)$$

and plugging $\mathcal{J}(\mathbf{q}, \mathbf{p})$ and $\mathcal{R}(\mathbf{q}, \mathbf{p})$ from (28) into the matching equations in (48), leads to

$$\mathbf{0} = \mathbf{J}_1 \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} - \mathbf{q}^\times \frac{\partial \mathcal{H}}{\partial \mathbf{p}} + \dot{\mathbf{q}} - \dot{\mathbf{q}}_e \quad (54)$$

$$\begin{aligned} \mathbf{B}(\mathbf{q})\mathbf{u} &= \mathbf{q}^\times \frac{\partial \mathcal{H}}{\partial \mathbf{q}} - \mathbf{J}_1^\top \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} + \mathbf{J}_2 \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} - \mathbf{p}^\times \frac{\partial \mathcal{H}}{\partial \mathbf{p}} \\ &\quad - \mathbf{K}_d \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} + \mathbf{D}(\mathbf{q}, \mathbf{p}) \frac{\partial \mathcal{H}}{\partial \mathbf{p}} + \dot{\mathbf{p}} - \dot{\mathbf{p}}_e. \end{aligned} \quad (55)$$

Assuming $\mathbf{M}_d(\mathbf{q}_e) = \mathbf{M}(\mathbf{q})$, (54) is satisfied if we choose $\mathbf{J}_1 = \begin{bmatrix} \mathbf{R}^\top & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \hat{\mathbf{r}}_{e1}^\top & \hat{\mathbf{r}}_{e2}^\top & \hat{\mathbf{r}}_{e3}^\top \end{bmatrix}^\top$. Indeed, we have $\dot{\mathbf{q}} = \mathbf{q}^\times \frac{\partial \mathcal{H}}{\partial \mathbf{p}}$ [from (11) and (19)] and

$$\frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} = \mathbf{M}_d^{-1} \mathbf{p}_e = \begin{bmatrix} \mathbf{v}_e \\ \boldsymbol{\omega}_e \end{bmatrix} = \begin{bmatrix} \mathbf{v} - \mathbf{R}^\top \mathbf{R}^* \mathbf{v}^* \\ \boldsymbol{\omega} - \mathbf{R}^\top \mathbf{R}^* \boldsymbol{\omega}^* \end{bmatrix}.$$

The error dynamics becomes

$$\dot{\mathbf{q}}_e = \begin{bmatrix} \dot{\mathbf{p}} - \dot{\mathbf{p}}^* \\ \dot{\mathbf{r}}_{e1} \\ \dot{\mathbf{r}}_{e2} \\ \dot{\mathbf{r}}_{e3} \end{bmatrix} = \begin{bmatrix} \mathbf{R}^\top & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \hat{\mathbf{r}}_{e1}^\top & \hat{\mathbf{r}}_{e2}^\top & \hat{\mathbf{r}}_{e3}^\top \end{bmatrix}^\top \begin{bmatrix} \mathbf{v}_e \\ \boldsymbol{\omega}_e \end{bmatrix} = \mathbf{J}_1 \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \quad (56)$$

since $\dot{\mathbf{R}}_e = \frac{d}{dt}(\mathbf{R}_e) = \mathbf{R}_e \hat{\omega}_e$ as shown in [79, Sec. III-A] and $\dot{\mathbf{p}} - \dot{\mathbf{p}}^* = \mathbf{R}\mathbf{v} - \mathbf{R}^*\mathbf{v}^* = \mathbf{R}\mathbf{v}_e$.

The desired control input can be obtained from (55) as $\mathbf{u} = \mathbf{u}_{ES} + \mathbf{u}_{DI}$ with

$$\mathbf{u}_{ES} = \mathbf{B}^\dagger(\mathbf{q}) \left(\mathbf{q}^{\times\top} \frac{\partial \mathcal{H}}{\partial \mathbf{q}} - \mathbf{J}_1^\top \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} + \mathbf{J}_2 \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} - \mathbf{p}^\times \frac{\partial \mathcal{H}}{\partial \mathbf{p}} + \mathbf{D}(\mathbf{q}, \mathbf{p}) \frac{\partial \mathcal{H}}{\partial \mathbf{p}} + \dot{\mathbf{p}} - \dot{\mathbf{p}}^* \right) \quad (57a)$$

$$\mathbf{u}_{DI} = -\mathbf{B}^\dagger(\mathbf{q}) \mathbf{K}_d \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \quad (57b)$$

where $\mathbf{B}^\dagger(\mathbf{q}) = (\mathbf{B}^\top(\mathbf{q})\mathbf{B}(\mathbf{q}))^{-1}\mathbf{B}^\top(\mathbf{q})$ is the pseudoinverse of $\mathbf{B}(\mathbf{q})$. The matching condition (50) becomes

$$\mathbf{B}^\perp(\mathbf{q}) \left(\mathbf{q}^{\times\top} \frac{\partial \mathcal{H}}{\partial \mathbf{q}} - \mathbf{J}_1^\top \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} + \mathbf{J}_2 \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} - \mathbf{p}^\times \frac{\partial \mathcal{H}}{\partial \mathbf{p}} + \dot{\mathbf{p}} - \dot{\mathbf{p}}^* \right) = 0. \quad (58)$$

In this article, we reshape the open-loop Hamiltonian $\mathcal{H}(\mathbf{q}, \mathbf{p})$ into the following desired total energy $\mathcal{H}_d(\mathbf{q}_e, \mathbf{p}_e)$, minimized along the desired trajectory:

$$\begin{aligned} \mathcal{H}_d(\mathbf{q}_e, \mathbf{p}_e) &= \frac{1}{2}(\mathbf{p} - \mathbf{p}^*)^\top \mathbf{K}_p(\mathbf{p} - \mathbf{p}^*) \\ &\quad + \frac{1}{2} \text{tr}(\mathbf{K}_R(\mathbf{I} - \mathbf{R}^{*\top} \mathbf{R})) \\ &\quad + \frac{1}{2}(\mathbf{p} - \mathbf{p}^*)^\top \mathbf{M}^{-1}(\mathbf{q})(\mathbf{p} - \mathbf{p}^*) \end{aligned} \quad (59)$$

where $\mathbf{K}_p, \mathbf{K}_R \succ 0$ are positive-definite matrices. For an SE(3) rigid-body system with constant generalized mass matrix $\mathbf{M}_d = \mathbf{M}$ and $\mathbf{J}_2 = 0$, which is a common choice, the energy-shaping term in (57a) and the damping-injection term in (57b) simplify as

$$\begin{aligned} \mathbf{u}_{ES}(\mathbf{q}, \mathbf{p}) &= \mathbf{B}^\dagger(\mathbf{q}) \left(\mathbf{q}^{\times\top} \frac{\partial V}{\partial \mathbf{q}} - (\mathbf{p}^\times - \mathbf{D}(\mathbf{q}, \mathbf{p})) \mathbf{M}^{-1} \mathbf{p} - \mathbf{e}(\mathbf{q}, \mathbf{q}^*) + \dot{\mathbf{p}}^* \right) \\ \mathbf{u}_{DI}(\mathbf{q}, \mathbf{p}) &= -\mathbf{B}^\dagger(\mathbf{q}) \mathbf{K}_d \mathbf{M}^{-1}(\mathbf{p} - \mathbf{p}^*) \end{aligned} \quad (60)$$

where the generalized coordinate error between $\mathbf{q}$ and $\mathbf{q}^*$ is

$$\mathbf{e}(\mathbf{q}, \mathbf{q}^*) := \mathbf{J}_1^\top \frac{\partial V_d}{\partial \mathbf{q}_e} = \left[\begin{array}{c} \mathbf{R}^\top \mathbf{K}_p(\mathbf{p} - \mathbf{p}^*) \\ \frac{1}{2}(\mathbf{K}_R \mathbf{R}^{*\top} \mathbf{R} - \mathbf{R}^\top \mathbf{R}^* \mathbf{K}_R^\top)^\vee \end{array} \right] \quad (61)$$

and the derivative of the desired momentum is

$$\dot{\mathbf{p}}^* = \mathbf{M} \left[\begin{array}{c} \mathbf{R}^\top \dot{\mathbf{p}}^* - \hat{\omega} \mathbf{R}^\top \mathbf{p}^* \\ \mathbf{R}^\top \mathbf{R}^* \dot{\omega}^* - \hat{\omega}_e \mathbf{R}^\top \mathbf{R}^* \omega^* \end{array} \right]. \quad (62)$$

By expanding the terms in (60), we have

$$\mathbf{p}^\times \mathbf{M}^{-1} \mathbf{p} = \mathbf{p}^\times \boldsymbol{\zeta} = \left[\begin{array}{c} \hat{\mathbf{p}}_v \omega \\ \hat{\mathbf{p}}_\omega \omega + \hat{\mathbf{p}}_v \mathbf{v} \end{array} \right] \quad (63)$$

$$\mathbf{M}^{-1}(\mathbf{p} - \mathbf{p}^*) = \left[\begin{array}{c} \mathbf{v} - \mathbf{R}^\top \dot{\mathbf{p}}^* \\ \omega - \mathbf{R}^\top \mathbf{R}^* \omega^* \end{array} \right] \quad (64)$$

$$\mathbf{q}^{\times\top} \frac{\partial \mathcal{V}}{\partial \mathbf{q}} = \left[\begin{array}{c} \mathbf{R}^\top \frac{\partial \mathcal{V}(\mathbf{q})}{\partial \mathbf{p}} \\ \sum_{i=1}^3 \hat{\mathbf{r}}_i^\top \frac{\partial \mathcal{V}(\mathbf{q})}{\partial \mathbf{r}_i} \end{array} \right]. \quad (65)$$

Theorem 1: Consider a port-Hamiltonian system on the SE(3) manifold with dynamics (30). Assume that the matching condition (58) is satisfied, the desired momentum's derivative $\dot{\mathbf{p}}^*$ is bounded, and the matrices $\mathbf{K}_p$, $\mathbf{K}_R$, and $\mathbf{K}_d$ are positive definite. The control policy in (57a) leads to closed-loop error dynamics in (47) and (53). The tracking error $(\mathbf{q}_e, \mathbf{p}_e) = ((\mathbf{p}_e, \mathbf{R}_e), \mathbf{p}_e)$ asymptotically stabilizes to $((\mathbf{0}, \mathbf{I}), \mathbf{0})$ with Lyapunov function given by the desired Hamiltonian $\mathcal{H}_d(\mathbf{q}_e, \mathbf{p}_e)$ in (59).

Proof: Since the matching condition is satisfied and the desired momentum's derivative $\dot{\mathbf{p}}^*$ is bounded, the control policy in (57a) and (57b) exists and achieves the desired closed-loop error dynamics as follows :

$$\left[\begin{array}{c} \dot{\mathbf{q}}_e \\ \dot{\mathbf{p}}_e \end{array} \right] = \left[\begin{array}{cc} \mathbf{0} & \mathbf{J}_1 \\ -\mathbf{J}_1^\top & \mathbf{J}_2 - \mathbf{K}_d \end{array} \right] \left[\begin{array}{c} \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e} \\ \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e} \end{array} \right]. \quad (66)$$

We have $\text{tr}(\mathbf{I} - \mathbf{R}^{*\top} \mathbf{R}) \geq 0$, as all entries in $\mathbf{R}_e \in SO(3)$ are less than 1. Since $\mathbf{M}$, $\mathbf{K}_p$, and $\mathbf{K}_R$ are positive-definite matrices, the desired Hamiltonian $\mathcal{H}_d$ is positive-definite, and achieves minimum value 0 only at $\mathbf{q}_e = (\mathbf{0}, \mathbf{I})$ and $\mathbf{p}_e = \mathbf{0}$, i.e., no position, rotation, and momentum errors. The time derivative of $\mathcal{H}_d(\mathbf{q}_e, \mathbf{p}_e)$ can be computed as

$$\begin{aligned} \dot{\mathcal{H}}_d(\mathbf{q}_e, \mathbf{p}_e) &= \frac{\partial \mathcal{H}_d}{\partial \mathbf{q}_e}^\top \dot{\mathbf{q}}_e + \frac{\partial \mathcal{H}_d}{\partial \mathbf{p}_e}^\top \dot{\mathbf{p}}_e \\ &= -\mathbf{p}_e^\top \mathbf{M}^{-1}(\mathbf{q}) \mathbf{K}_d \mathbf{M}^{-1}(\mathbf{q}) \mathbf{p}_e. \end{aligned} \quad (67)$$

As $\mathbf{K}_d$ and $\mathbf{M}(\mathbf{q})$ are positive-definite, we have $\dot{\mathcal{H}}_d(\mathbf{q}_e, \mathbf{p}_e) \leq 0$ for all $(\mathbf{q}_e, \mathbf{p}_e)$ and equality holds at $((\mathbf{0}, \mathbf{I}), \mathbf{0})$. By LaSalle's invariance principle [80], the tracking errors $(\mathbf{q}_e, \mathbf{p}_e)$ asymptotically converge to $((\mathbf{0}, \mathbf{I}), \mathbf{0})$. $\square$

Without requiring a priori knowledge of the system parameters, the control design in (57a) offers a unified control approach for SE(3) Hamiltonian systems that achieves trajectory tracking, if permissible by the system's degree of underactuation. Thus, our control design solves Problem 2 for rigid-body robot systems, such as UGVs, UAVs, and UUVs, with tracking performance guaranteed by Theorem 1.

VII. EVALUATION

We verify the effectiveness of our port-Hamiltonian neural ODE network for dynamics learning and control on matrix Lie groups using a simulated pendulum, a simulated Crazyflie quadrotor, and a real PX4 quadrotor platform, whose states evolve on the SE(3) manifold. The implementation details for the experiments are provided in Appendix A.

A. Pendulum

In this section, we verify our port-Hamiltonian dynamics learning and control approach on the $SO(3)$ manifold. We consider a pendulum with the following dynamics:

$$\ddot{\varphi} = -15 \sin \varphi + 3u - 0.2\dot{\varphi} \quad (68)$$

where φ is the angle of the pendulum with respect to its vertically downward position, and u is a scalar control input. The ground-truth mass, potential energy, friction coefficient, and the input gain are: $m = 1/3$, $\mathcal{V}(\varphi) = 5(1 - \cos \varphi)$, $D(\varphi) = 0.2/3$, and $B(\varphi) = 1$, respectively. We collected data of the form $\{(\cos \varphi, \sin \varphi, \dot{\varphi})\}$ from an OpenAI Gym environment, provided by [27], with the dynamics in (68). To illustrate our Lie group neural ODE learning, we represent the angle φ as a rotation matrix

$$\mathbf{R} = \begin{bmatrix} \cos \varphi & -\sin \varphi & 0 \\ \sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (69)$$

representing the pendulum orientation. We let $\boldsymbol{\omega} = [0, 0, \dot{\varphi}]$ and remove position $\mathbf{p}$ and linear velocity $\mathbf{v}$ from the Hamiltonian dynamics in (30), restricting the system to the $SO(3)$ manifold with generalized coordinates $\mathbf{q} = [\mathbf{r}_1^\top \ \mathbf{r}_2^\top \ \mathbf{r}_3^\top]^\top$.

As described in Section V-D, control inputs $\mathbf{u}^{(i)}$ were sampled randomly and applied to the pendulum for five time intervals of $0.05s$, forming a dataset $\mathcal{D} = \{t_{0:N}^{(i)}, \mathbf{q}_{0:N}^{(i)}, \boldsymbol{\omega}_{0:N}^{(i)}, \mathbf{u}^{(i)}\}_{i=1}^D$ with $N = 5$ and $D = 5120$. We trained an $SO(3)$ port-Hamiltonian neural ODE network as described in Section V-D for 5000 iterations without any nominal model, i.e., $\mathbf{M}_{\omega_0}^{-1}(\mathbf{q}) = \mathbf{0}$, $\mathbf{D}_{\omega_0}(\mathbf{q}, \mathbf{p}) = \mathbf{0}$, $V_0(\mathbf{q}) = 0$, and $\mathbf{B}_0(\mathbf{q}) = \mathbf{0}$.

As noted in [27] and [32], since the generalized momenta $\mathbf{p}$ are not available in the dataset, the dynamics of $\mathbf{q}$ in (68) do not change if $\mathbf{p}$ is scaled by a factor $\beta > 0$. This is also true in our formulation as scaling $\mathbf{p}$ leaves the dynamics of $\mathbf{q}$ in (30) unchanged. To emphasize this scale invariance, let $\mathbf{M}_\beta(\mathbf{q}) = \beta \mathbf{M}(\mathbf{q})$, $\mathcal{V}_\beta(\mathbf{q}) = \beta \mathcal{V}(\mathbf{q})$, $\mathbf{D}_\beta(\mathbf{q}, \mathbf{p}) = \beta \mathbf{D}(\mathbf{q}, \mathbf{p})$, $\mathbf{B}_\beta(\mathbf{q}) = \beta \mathbf{B}(\mathbf{q})$, and

$$\begin{aligned} \mathbf{p}_\beta &= \mathbf{M}_\beta(\mathbf{q})\boldsymbol{\omega} = \beta \mathbf{p}, & \dot{\mathbf{p}}_\beta &= \beta \dot{\mathbf{p}} \\ \mathcal{H}_\beta(\mathbf{q}, \mathbf{p}) &= \frac{1}{2} \mathbf{p}_\beta^\top \mathbf{M}_\beta^{-1}(\mathbf{q}) \mathbf{p}_\beta + V_\beta(\mathbf{q}) = \beta \mathcal{H}(\mathbf{q}, \mathbf{p}) \\ \frac{\partial \mathcal{H}_\beta(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}_\beta} &= \mathbf{M}_\beta^{-1}(\mathbf{q}) \mathbf{p}_\beta = \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{p}} \end{aligned} \quad (70)$$

guaranteeing that the equations of motions (30) still hold.

Fig. 3 shows the training and testing behavior of our $SO(3)$ Hamiltonian ODE network. Fig. 3(a) and (c) shows that the $[\mathbf{M}(\mathbf{q})^{-1}]_{3,3}$ entry of the mass inverse and the $[\mathbf{B}(\mathbf{q})]_3$ entry of the input matrix with scaling factor $\beta = 1.33$ are close to their correct values of 3 and 1, respectively, while the other entries are close to zero. Fig. 3(b) indicates a constant gap between the learned and the ground-truth potential energy, which can be explained by the relativity of potential energy.

We tested stabilization of the pendulum based on the learned dynamics to the unstable equilibrium at the upward position $\varphi = \pi$, with zero velocity. Since the pendulum is a fully actuated

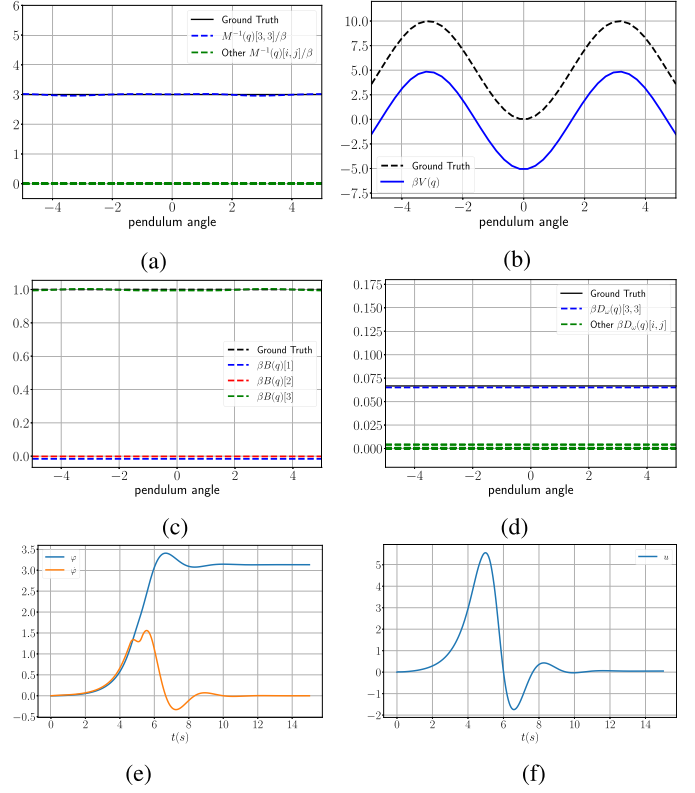


Fig. 3. Pendulum dynamics estimation using an $SO(3)$ port-Hamiltonian neural ODE network with scale factor $\beta = 1.33$. (a) $\mathbf{M}^{-1}(\mathbf{q})/\beta$ versus φ . (b) $\beta \mathcal{V}(\mathbf{q})$ versus φ . (c) $\beta \mathbf{B}(\mathbf{q})$ versus φ . (d) $\beta \mathbf{D}_\omega(\mathbf{q})$ versus φ . (e) Angle φ and velocity $\dot{\varphi}$. (f) Control input u .

system, the energy-based controller in (60) exists and is obtained by removing the position error from the desired energy

$$\mathcal{H}_d(\mathbf{q}_e, \mathbf{p}_e) = \frac{1}{2} \text{tr}(\mathbf{K}_R(\mathbf{I} - \mathbf{R}^{*\top} \mathbf{R})) + \frac{1}{2} \mathbf{p}_\omega^\top \mathbf{M}^{-1}(\mathbf{q}) \mathbf{p}_\omega. \quad (71)$$

The controlled angle φ and angular velocity $\dot{\varphi}$ as well as the control inputs $\mathbf{u}$ with gains $\mathbf{K}_R = 2\mathbf{I}$ and $\mathbf{K}_d = \mathbf{I}$ are shown over time in Fig. 3(e) and (f). We can see that the controller was able to smoothly drive the pendulum from $\phi = 0$ to $\phi = \pi$, relying only on the learned dynamics.

B. Omnidirectional Ground Vehicle

In this section, we verify our port-Hamiltonian dynamics learning and control approach on a simulated omnidirectional ground vehicle, whose states evolve on the $SE(2)$ manifold. The ground-truth dynamics of the vehicle can be obtained from (30) by keeping only the components x and y in the position $\mathbf{p}$ and the yaw angle of the rotation matrix, leading to an $SO(2)$ rotation matrix

$$\mathbf{R} = \begin{bmatrix} \cos \varphi & -\sin \varphi \\ \sin \varphi & \cos \varphi \end{bmatrix} \quad (72)$$

where φ is the vehicle's yaw angle. The vehicle moves on a flat ground with potential energy $\mathcal{V}(\mathbf{q}) = c$, where c is a

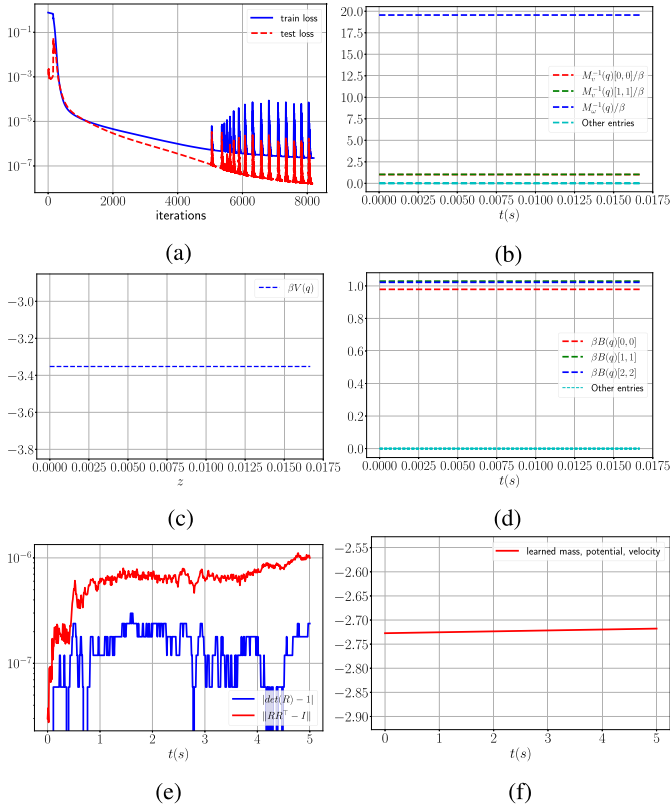


Fig. 4. Evaluation of SE(2) port-Hamiltonian neural ODE network on a simulated omnidirectional ground vehicle with scale factor $\beta = 7.1$. (a) Loss (log scale). (b) $M_v^{-1}(\mathbf{q})$ and $M_\omega^{-1}(\mathbf{q})$. (c) $V(\mathbf{q})$. (d) $B_v(\mathbf{q})$. (e) SO(3) constraints. (f) Total energy.

constant, and has ground-truth mass $M_v(\mathbf{q}) = \mathbf{I} \in \mathbb{S}_{>0}^{2 \times 2}$ and inertia $M_\omega(\mathbf{q}) = 0.05 \in \mathbb{R}_{>0}$. It is fully actuated with control input $\mathbf{u} = [f_x, f_y, \tau_\varphi]$, where f_x and f_y are forces along the x - and y -axes of the body frame and τ_φ is the yaw torque, generated by the motors.

To collect training data, the vehicle was controlled from a random initial point to nine different desired positions and yaw angles using a PID controller, providing nine one-second trajectories. The trajectories were used to generate a dataset $\mathcal{D} = \{t_{0:N}^{(i)}, \mathbf{q}_{0:N}^{(i)}, \zeta_{0:N}^{(i)}, \mathbf{u}^{(i)}\}_{i=1}^D$ with $N = 5$ and $D = 432$. The SE(2) port-Hamiltonian ODE network was formulated, as described in Section V-D, by ignoring the z component of the position $\mathbf{p}$, and the pitch and roll components of the rotation $\mathbf{R}$. The model was trained for 8000 iterations without a nominal model, i.e., $M_{v0}^{-1}(\mathbf{q}) = \mathbf{0}$, $M_{\omega0}^{-1}(\mathbf{q}) = \mathbf{0}$, $D_{v0}(\mathbf{q}, \mathbf{p}) = \mathbf{0}$, $D_{\omega0}(\mathbf{q}, \mathbf{p}) = \mathbf{0}$, $V_0(\mathbf{q}) = \mathbf{0}$, and $B_0(\mathbf{q}) = \mathbf{0}$. We did not consider energy dissipation such as friction in the simulation, and we omitted the dissipation matrix $D_\theta(\mathbf{q}, \mathbf{p})$ in the model.

Fig. 4 shows the training results for the SE(2) Hamiltonian ODE network. Fig. 4(b) and (d) shows that the mass inverse and the input gain matrix with scaling factor $\beta = 7.1$ are close to their correct values: $M_v(\mathbf{q})^{-1} \approx \mathbf{I}$, $M_\omega(\mathbf{q})^{-1} \approx 20$ and $B(\mathbf{q}) \approx \mathbf{I}$. Fig. 4(c) shows a constant learned potential energy as expected.

We verified our energy-based control design in Section VI by controlling the ground robot to track horizontal lemniscate

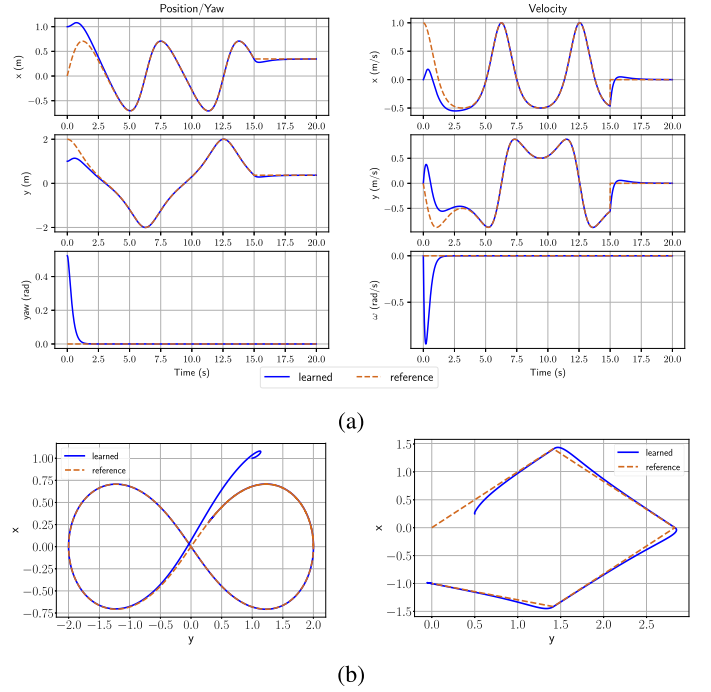


Fig. 5. Trajectories (blue) of omnidirectional ground vehicle tracking desired trajectories (orange) with our learned SE(2) port-Hamiltonian dynamics and IDA-PBC control design. (a) Tracking performance with a lemniscate trajectory. (b) Tracking lemniscate (left) and piecewise-linear (right) trajectories.

and piecewise-linear trajectories. Fig. 5(b) demonstrates that the omnidirectional ground vehicle controlled by our energy-based controller achieves successful trajectory tracking. The control gains were chosen as: $\mathbf{K}_p = 0.72\mathbf{I}$, $\mathbf{K}_v = 0.8\mathbf{I}$, $\mathbf{K}_R = 9.1\mathbf{I}$, and $\mathbf{K}_\omega = 3.6$. Fig. 5(a) plots the tracking errors in the position, yaw angles, linear velocity, and angular velocity.

C. Crazyflie Quadrotor

In this section, we demonstrate that our SE(3) dynamics learning and control approach can achieve trajectory tracking for an underactuated system. We consider a Crazyflie quadrotor, shown in Fig. 8(a), simulated in the physics-based simulator PyBullet [81]. The control input $\mathbf{u} = [f, \boldsymbol{\tau}]$ includes a thrust $f \in \mathbb{R}_{\geq 0}$ and a torque vector $\boldsymbol{\tau} \in \mathbb{R}^3$ generated by the four rotors. The generalized coordinates and velocity are $\mathbf{q} = [\mathbf{p}^\top, \mathbf{r}_1^\top, \mathbf{r}_2^\top, \mathbf{r}_3^\top]^\top$ and $\zeta = [\mathbf{v}^\top, \boldsymbol{\omega}^\top]^\top$ as before.

The quadrotor was controlled from a random starting point to 18 different desired poses using a PID controller [81], providing 18 2.5-s trajectories. The trajectories were used to generate a dataset $\mathcal{D} = \{t_{0:N}^{(i)}, \mathbf{q}_{0:N}^{(i)}, \zeta_{0:N}^{(i)}, \mathbf{u}^{(i)}\}_{i=1}^D$ with $N = 5$ and $D = 1080$. The SE(3) port-Hamiltonian ODE network was trained, as described in Section V-D, for 500 iterations without a nominal model, i.e., $M_{v0}^{-1}(\mathbf{q}) = \mathbf{0}$, $M_{\omega0}^{-1}(\mathbf{q}) = \mathbf{0}$, $D_{v0}(\mathbf{q}, \mathbf{p}) = \mathbf{0}$, $D_{\omega0}(\mathbf{q}, \mathbf{p}) = \mathbf{0}$, $V_0(\mathbf{q}) = \mathbf{0}$, and $B_0(\mathbf{q}) = \mathbf{0}$. We did not consider energy dissipation such as drag effect in the PyBullet simulator, and we omitted the dissipation matrix $D_\theta(\mathbf{q}, \mathbf{p})$ in the model design.

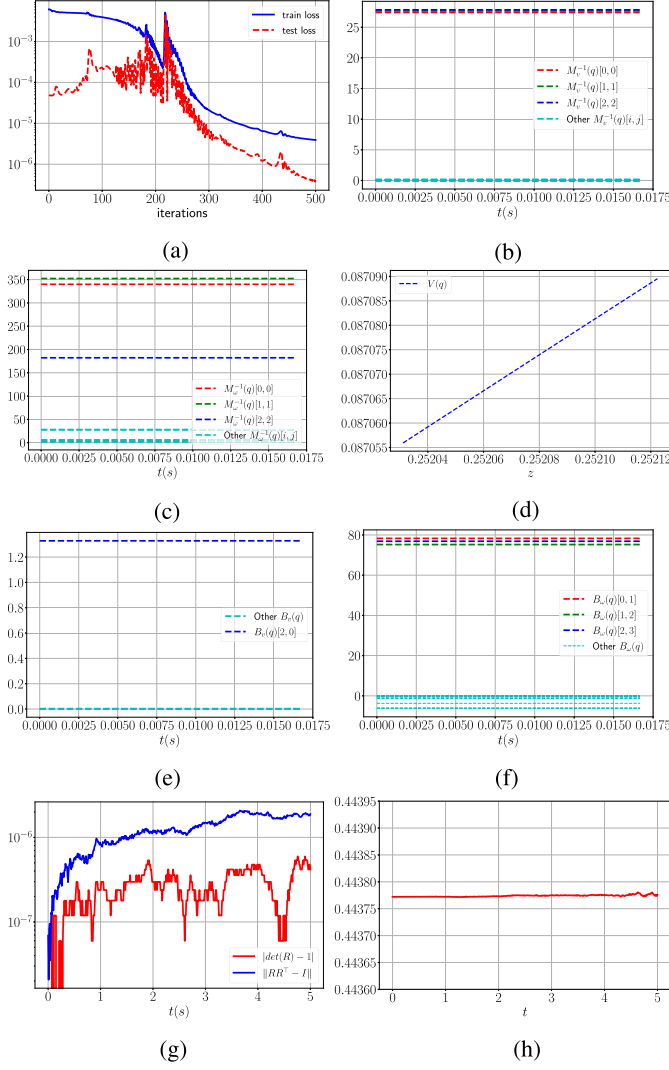


Fig. 6. SE(3) port-Hamiltonian neural ODE network on a Crazyflie quadrotor in the PyBullet simulator [81]. (a) Loss (log scale). (b) $M_v^{-1}(\mathbf{q})$. (c) $M_\omega^{-1}(\mathbf{q})$. (d) $V(\mathbf{q})$. (e) $B_v(\mathbf{q})$. (f) $B_\omega(\mathbf{q})$. (g) SO(3) constraints. (h) Total energy.

The training and test results are shown in Fig. 6. The learned generalized mass and inertia converged to constant diagonal matrices: $M_v^{-1}(\mathbf{q}) \approx 27.5\mathbf{I}$ and $M_\omega^{-1}(\mathbf{q}) \approx \text{diag}([351, 340, 181])$. The input matrix $B_v(\mathbf{q})$ converged to a constant matrix with $[B_v(\mathbf{q})]_{2,0} \approx 1.32$, while the other entries were closed to 0, consistent with the fact that the quadrotor thrust only affects the linear velocity along the z -axis in the body-fixed frame. The input matrix $B_\omega(\mathbf{q})$ converged to $\sim 76\mathbf{I}$ as the motor torques affects all components of the angular velocity ω . The learned potential energy $V(\mathbf{q})$ was linear in the height z , agreeing with the gravitational potential.

We also verified our energy-based control design in Section VI by controlling the quadrotor to track a desired trajectory based on the learned dynamics model. Given desired position $\mathbf{p}^*$ and heading ψ^* (yaw angle), we construct an appropriate $\mathbf{R}^*$ and $\mathbf{p}^*$ to be used with the energy-based controller in (60). By expanding the terms in (60) and choosing the control gain $\mathbf{K}_d$ of the form

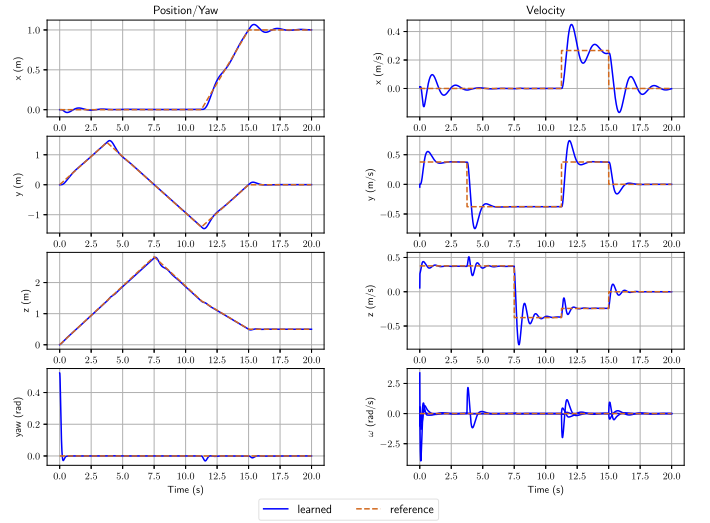


Fig. 7. Crazyflie quadrotor trajectory (blue) tracking a desired diamond-shaped trajectory (orange) shown in Fig. 8.

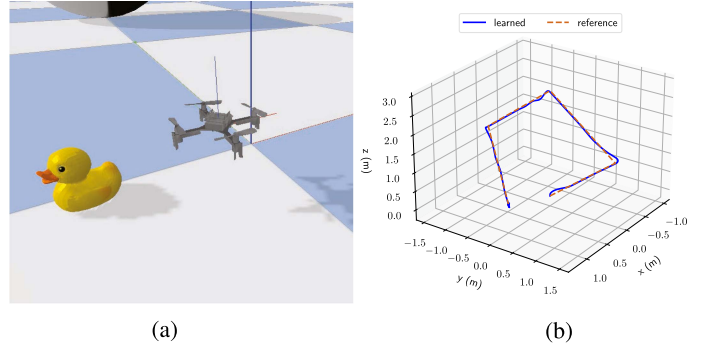


Fig. 8. Trajectory tracking experiment with a Crazyflie quadrotor in the PyBullet simulator [81]. (a) Crazyflie simulator. (b) Trajectory tracking.

TABLE I
COMPARISON OF DIFFERENT NEURAL NETWORK ARCHITECTURES FOR PENDULUM AND QUADROTOR DYNAMICS LEARNING

Metrics	Platform	Black-box	Unstructured Hamiltonian	Structured Hamiltonian
Training loss	Pendulum	0.037	6.3×10^{-5}	2.3×10^{-7}
$\ \det(\mathbf{R}) - 1\ $ (avg.)	Pendulum	1888752	1.4×10^{-3}	1.4×10^{-3}
$\ \mathbf{R}\mathbf{R}^T - \mathbf{I}\ $ (avg.)	Pendulum	267971.6	2.1×10^{-3}	2.1×10^{-3}
Total energy (std.)	Pendulum	13557.1	0.163	0.003
Prediction error (avg.)	Pendulum	1.02 (rad)	0.08 (rad)	0.008 (rad)
Training loss	Quadrotor	2.2×10^{-3}	6.4×10^{-4}	3.92×10^{-6}
$\ \det(\mathbf{R}) - 1\ $ (avg.)	Quadrotor	3741	2.9×10^{-6}	2.6×10^{-7}
$\ \mathbf{R}\mathbf{R}^T - \mathbf{I}\ $ (avg.)	Quadrotor	29336.7	7.6×10^{-6}	1.3×10^{-6}
Total energy (std.)	Quadrotor	18.1	0.074	1.64×10^{-6}
Prediction error (avg.)	Quadrotor	0.49 (m)	0.46 (m)	0.02 (m)

The bold values are to emphasize that our method outperforms other baselines, i.e. achieving smaller errors.

$\mathbf{K}_d = \begin{bmatrix} \mathbf{K}_v & \mathbf{0} \\ \mathbf{0} & \mathbf{K}_\omega \end{bmatrix}$, the control input can be written explicitly as

$$\mathbf{u} = \mathbf{B}^\dagger(\mathbf{q}) \begin{bmatrix} \tau_v \\ \tau_\omega \end{bmatrix} \quad (73)$$

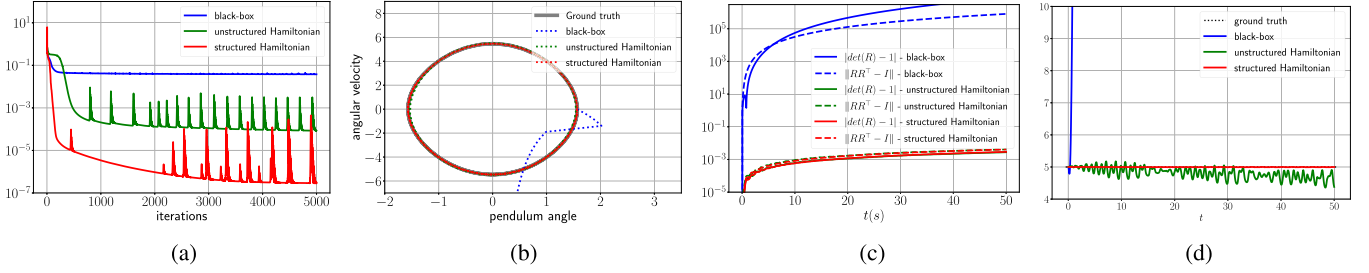


Fig. 9. Comparison of different neural network architectures to learn pendulum dynamics: black-box, i.e., the dynamics function $\mathbf{f}$ is represented by a multilayer perceptron network; unstructured Hamiltonian, i.e., the Hamiltonian function is represented by a multilayer perceptron network instead of the sum of kinetic and potential energy as shown in (27); and structured Hamiltonian, i.e., the Hamiltonian function has the form of (27). Initialized at $\phi = \pi/2$, the learned pendulum dynamics are rolled out, showing that our approach with structured Hamiltonian preserves the phase portraits, SO(3) constraints, and the conservation of energy better than the other models. (a) Training loss. (b) Phase portraits. (c) SO(3) constraints. (d) Total energy.

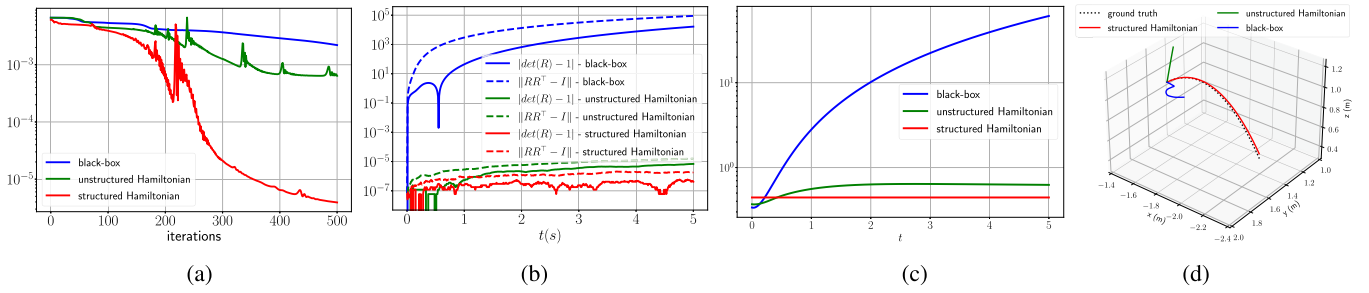


Fig. 10. Comparison of different neural network architectures to learn quadrotor dynamics: black-box, i.e., the dynamics function $\mathbf{f}$ is represented by a multilayer perceptron network; unstructured Hamiltonian, i.e., the Hamiltonian function is represented by a multilayer perceptron network instead of the sum of kinetic and potential energy as shown in (27); and structured Hamiltonian, i.e., the Hamiltonian function has the form of (27). Initialized at a random pose and twist, the learned quadrotor dynamics are rolled out, showing that our approach with structured Hamiltonian preserves SO(3) constraints and the conservation of energy better than the other models, and provides better state predictions. (a) Training loss. (b) SO(3) constraints. (c) Total energy. (d) Predicted trajectories.

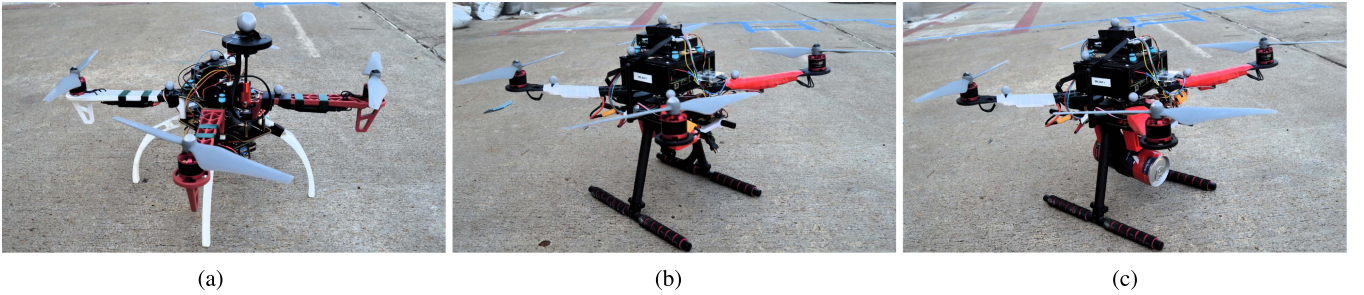


Fig. 11. Quadrotor robots used in the experiments. (a) RaspberryPi quadrotor whose mass and inertia matrix serve as nominal values for our learning framework. (b) Intel NUC quadrotor with a different frame. (c) Intel NUC quadrotor carrying a coffee can as payload.

where

$$\begin{aligned} \tau_v = \mathbf{R}^\top \frac{\partial \mathcal{V}(\mathbf{q})}{\partial \mathbf{p}} - \hat{\mathbf{p}}_v \omega - \mathbf{R}^\top \mathbf{K}_p (\mathbf{p} - \mathbf{p}^*) \\ - \mathbf{K}_v (\mathbf{v} - \mathbf{R}^\top \dot{\mathbf{p}}^*) + \mathbf{M}_1 (\mathbf{R}^\top \ddot{\mathbf{p}}^* - \dot{\omega} \mathbf{R}^\top \dot{\mathbf{p}}^*) \end{aligned} \quad (74)$$

$$\begin{aligned} \tau_\omega = \sum_{i=1}^3 \hat{\mathbf{r}}_i \frac{\partial \mathcal{V}(\mathbf{q})}{\partial \mathbf{r}_i} - \mathbf{K}_\omega (\omega - \mathbf{R}^\top \mathbf{R}^* \omega^*) \\ - (\hat{\mathbf{p}}_\omega \omega + \hat{\mathbf{p}}_v \mathbf{v}) - \frac{1}{2} (\mathbf{K}_R \mathbf{R}^{*\top} \mathbf{R} - \mathbf{R}^\top \mathbf{R}^* \mathbf{K}_R^\top)^\vee \\ + \mathbf{M}_2 (\mathbf{R}^\top \mathbf{R}^* \dot{\omega}^* - \dot{\omega}_e \mathbf{R}^\top \mathbf{R}^* \omega^*). \end{aligned} \quad (75)$$

Note that $\tau_v \in \mathbb{R}^3$ is the desired thrust in the body frame and depends only on the desired position $\mathbf{p}^*$ and the current pose. The desired thrust is transformed to the world frame as $\mathbf{R} \tau_v$. Inspired by [79], the vector $\mathbf{R} \tau_v$ should be along the z -axis of the body frame, i.e., the third column $\mathbf{r}_3^*$ of the desired rotation matrix $\mathbf{R}^*$. The second column $\mathbf{r}_2^*$ of the desired rotation matrix $\mathbf{R}^*$ can be chosen so that it has the desired yaw angle ψ^* and is perpendicular to $\mathbf{r}_3^*$. This can be done by projecting the second column of the yaw's rotation matrix $\mathbf{r}_2^\psi = [-\sin \psi, \cos \psi, 0]$ onto the plane perpendicular to $\mathbf{r}_3^*$. We have $\mathbf{R}^* = [\mathbf{r}_1^* \ \mathbf{r}_2^* \ \mathbf{r}_3^*]$ where

$$\mathbf{r}_3^* = \frac{\mathbf{R} \tau_v}{\|\mathbf{R} \tau_v\|}, \mathbf{r}_1^* = \frac{\mathbf{r}_2^\psi \times \mathbf{r}_3^*}{\|\mathbf{r}_2^\psi \times \mathbf{r}_3^*\|}, \mathbf{r}_2^* = \mathbf{r}_3^* \times \mathbf{r}_1^* \quad (76)$$

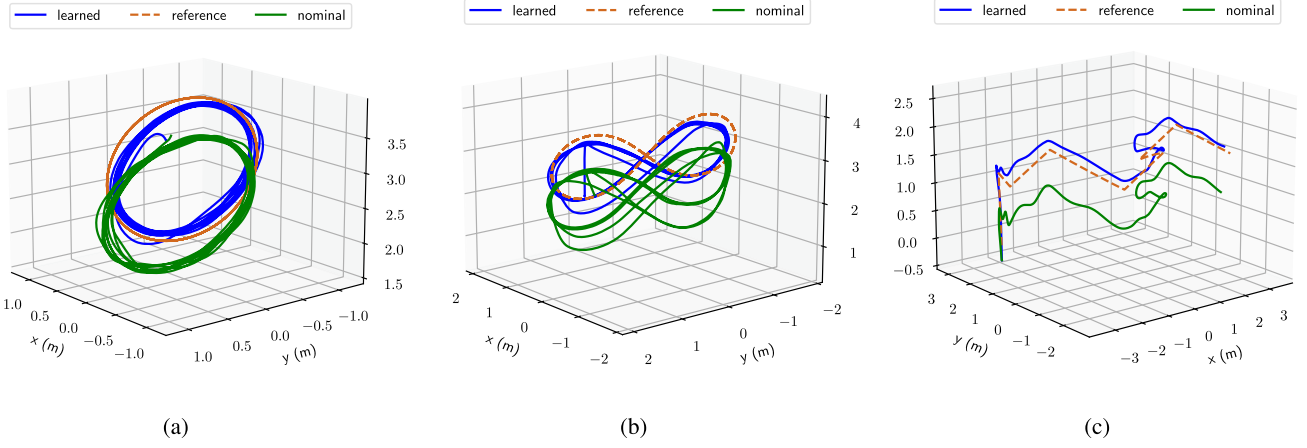


Fig. 12. Trajectory tracking with real quadrotors. (a) Vertical circle. (b) Vertical lemniscate. (c) Piecewise linear trajectory.

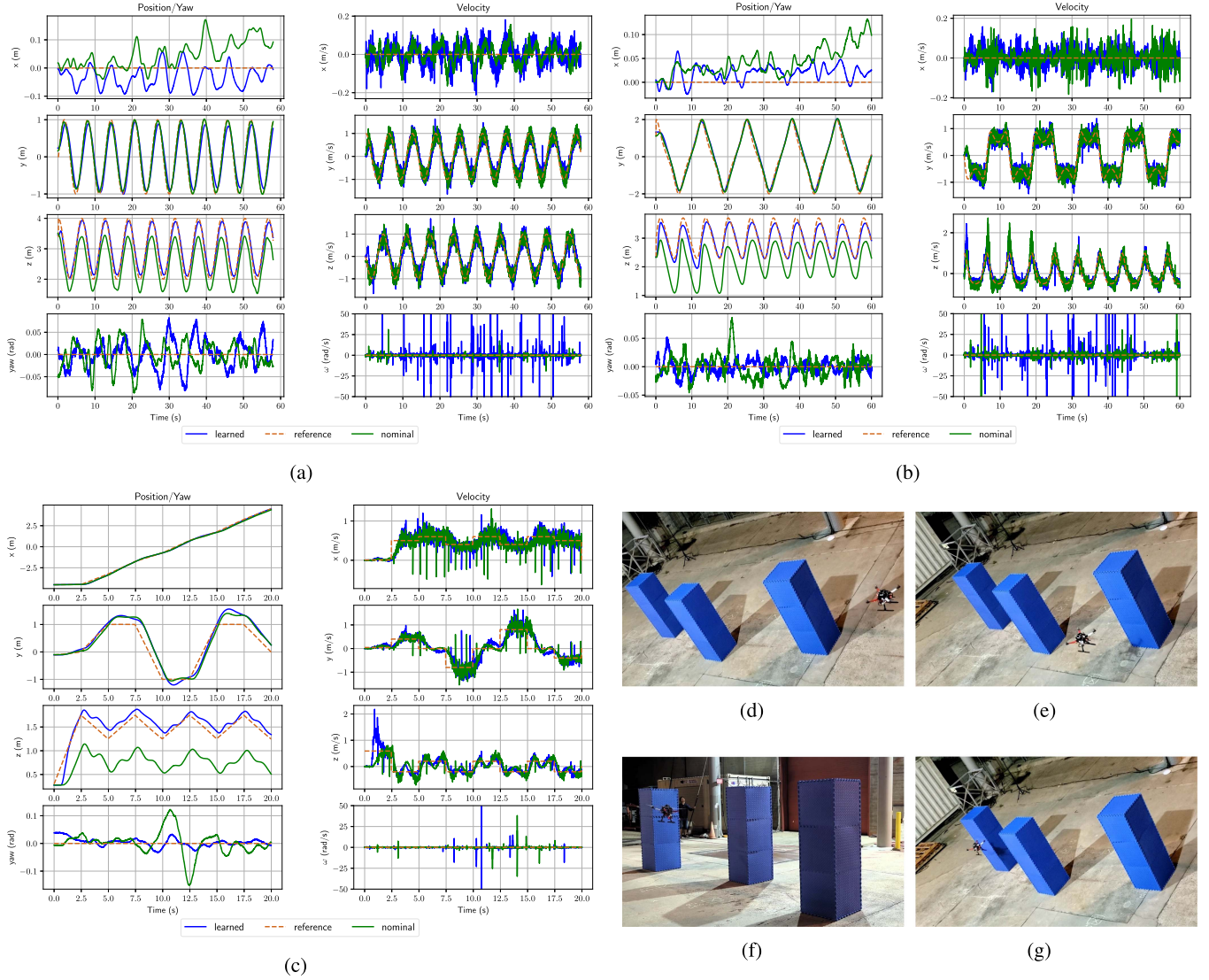


Fig. 13. Real quadrotor trajectory using our learned model and controller (blue) and using a nominal model and a geometric controller [79] (green) tracking a desired trajectory. (a) Vertical circle. (b) Vertical lemniscate. (c) Piecewise linear. (d) Tracking at $t = 0$ s. (e) Tracking at $t = 3$ s. (f) Tracking at $t = 9$ s. (g) Tracking at $t = 12$ s.

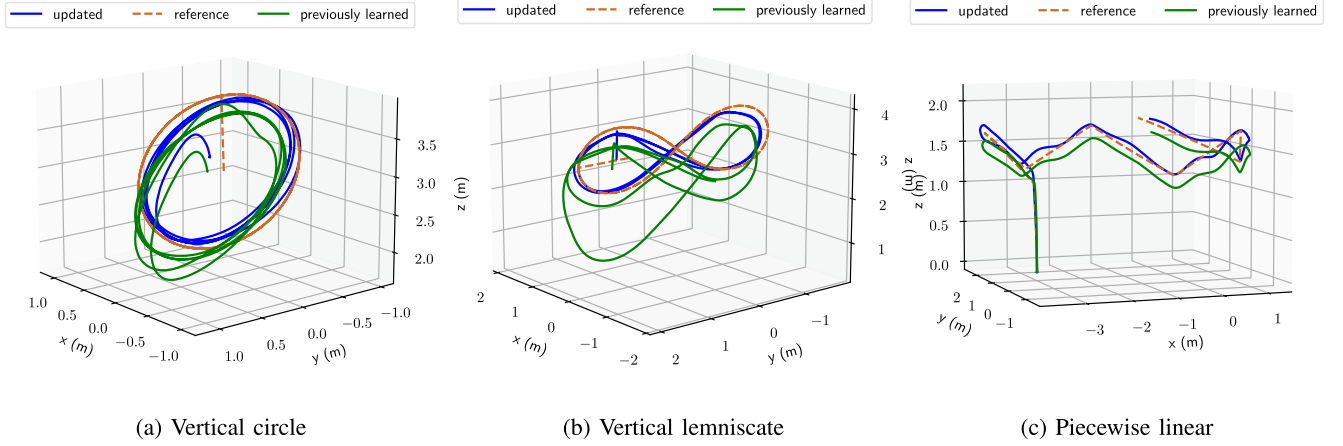


Fig. 14. Quadrotor trajectory tracking experiment with extra payload: (a) vertical circle, (b) vertical lemniscate, and (c) piecewise linear trajectory.

and $\dot{\omega}^* = \mathbf{R}^{*\top} \dot{\mathbf{R}}^*$. The derivative $\dot{\mathbf{R}}^*$ is calculated as follows:

$$\dot{\mathbf{r}}_3^* = \mathbf{r}_3^* \times \frac{\mathbf{R}^* \dot{\boldsymbol{\tau}}_v}{\|\mathbf{R}^* \dot{\boldsymbol{\tau}}_v\|} \times \mathbf{r}_3^* \quad (77)$$

$$\dot{\mathbf{r}}_1^* = \mathbf{r}_1^* \times \frac{\dot{\mathbf{r}}_2^* \times \mathbf{r}_3^* + \mathbf{r}_2^* \times \dot{\mathbf{r}}_3^*}{\|\mathbf{r}_2^* \times \mathbf{r}_3^*\|} \times \mathbf{r}_1^* \quad (78)$$

$$\dot{\mathbf{r}}_2^* = \dot{\mathbf{r}}_3^* \times \mathbf{r}_1^* + \mathbf{r}_3^* \times \dot{\mathbf{r}}_1^*. \quad (79)$$

Plugging $\mathbf{R}^*$ and ω^* back in τ_ω , we obtain the complete control input $\mathbf{u}$ in (73).

Fig. 8(b) shows qualitatively that the quadrotor controlled by our energy-based controller achieves successful trajectory tracking. The control gains were chosen as: $\mathbf{K}_p = \text{diag}([0.8, 0.8, 3.9])$, $\mathbf{K}_v = 0.23\mathbf{I}$, $\mathbf{K}_R = \text{diag}([3.6, 3.6, 6.9])$, and $\mathbf{K}_\omega = \text{diag}([0.3, 0.3, 0.6])$. Fig. 7 shows quantitatively the tracking errors in the position, yaw angles, linear velocity, and angular velocity. On an Intel i9 3.1-GHz CPU with 32-GB RAM, our controller's computation time in Python 3.7 was about 2.5 ms per control input, including forward passes through the neural networks, showing that it is suitable for fast real-time applications.

D. Comparison to Unstructured Neural ODE Models

In this section, we show the benefits of our neural ODE architecture by comparing the following: 1) our *structured Hamiltonian model*; 2) a *black-box model*, i.e., the approximated dynamics $\mathbf{f}$ is represented by a multilayer perceptron network; and 3) an *unstructured Hamiltonian model*, i.e., the Hamiltonian function is represented by a multilayer perceptron network instead of using the structure in (27), in terms of training convergence rates, satisfaction of energy conservation principle, and Lie group constraints. To verify energy conservation, we rolled out the learned dynamics and calculated the Hamiltonian via (27) along the predicted trajectories for the following:

- 1) the black-box model using ground-truth mass and potential energy with the predicted states;
- 2) the unstructured Hamiltonian model using the output of the multilayer perceptron Hamiltonian network;

- 3) the structured Hamiltonian model using the learned mass and potential energy networks.

We check the $\text{SO}(3)$ constraints by verifying that two quantities $|\det \mathbf{R} - 1|$ and $\|\mathbf{R}\mathbf{R}^\top - \mathbf{I}\|$ remain small along the predicted trajectories.

We first use a pendulum as described in Section VII-A without energy dissipation. The models are trained for 5000 iterations from 512 0.2-s state-control trajectories and rolled out for a significantly longer horizon of 50 s. Fig. 9 plots the training loss, the phase portraits, the $\text{SO}(3)$ constraints, and the total energy (Hamiltonian) of the learned models for a pendulum system. As the Hamiltonian structure is imposed in the neural ODE network architecture, our model is able to converge faster with lower loss [see Fig. 9(a) and Table I], preserves the phase portraits for state predictions [see Fig. 9(b)], and achieves the lowest angle prediction error in Table I. Fig. 9(c) and Table I show that the $\text{SO}(3)$ constraints are satisfied by our structured and unstructured Hamiltonian models as their values of $|\det \mathbf{R} - 1|$ and $\|\mathbf{R}\mathbf{R}^\top - \mathbf{I}\|$ remain small along a 50-s trajectory rollout initialized at $\phi = \pi/2$. The constant Hamiltonian in Fig. 9(d) of our structured Hamiltonian with lowest standard derivation value in Table I verifies that our model obeys the energy conservation law with high precision, given no control input and no energy dissipation. The Hamiltonian of the black-box model increases along the trajectory while that of the unstructured Hamiltonian model fluctuates and slightly decreases over time.

We also tested the models using the simulated Crazyflie quadrotor with the same dataset $\mathcal{D}$ of 18 trajectories as described in Section VII-C. The $\text{SE}(3)$ port-Hamiltonian ODE network was trained, as described in Section V-D, for 500 iterations. Our structured Hamiltonian model converges faster with significantly lower loss as seen in Fig. 10(a) and in Table I. We verified that the predicted orientation trajectories from our learned models satisfy the $\text{SO}(3)$ constraints. Fig. 10(b) and Table I show two near-zero quantities $|\det \mathbf{R} - 1|$ and $\|\mathbf{R}\mathbf{R}^\top - \mathbf{I}\|$, obtained by rolling out our learned dynamics for 5 s, while the learned black-box model significantly violates the constraints after a very short time. Fig. 10(c) shows a constant total energy

along the predicted trajectory, i.e., lowest standard derivation value in Table I, from our structured Hamiltonian model without control input and dissipation networks, verifying that the learned model obeys the law of energy conservation. Fig. 10(d) and the prediction error in Table I show that our structured Hamiltonian model provides better trajectory predictions compared to the other methods.

E. Real Quadrotor Experiments

In this section, we verify our approach using a real quadrotor robot, equipped with an onboard i7 Intel NUC computer and a PX4 flight controller [see Fig. 11(b)]. The quadrotor's pose and twist were provided by a motion capture system.

1) *Learning Quadrotor Dynamics After Upgrade:* We consider a scenario in which the quadrotor is upgraded with a new frame and a new onboard computer, leading to changes in the robot dynamics that we aim to learn from data. The nominal model was obtained from a computer-aided design (CAD) model of another much lighter weight Raspberry Pi quadrotor with a Raspberry Pi computer and an F450 frame [see Fig. 11(a)], which is less accurate and far from the unknown ground-truth model of our upgraded quadrotors. Specifically, the nominal mass and inertia matrix are $\mathbf{M}_{v0} = 1.3\mathbf{I}$ and $\mathbf{M}_{\omega0} = \text{diag}([0.12, 0.12, 0.2])$, respectively, for the upgraded quadrotor in Fig. 11(b). The other nominal matrices were set to zero: $\mathbf{D}_{v0}(\mathbf{q}, \mathbf{p}) = \mathbf{0}$, $\mathbf{D}_{\omega0}(\mathbf{q}, \mathbf{p}) = \mathbf{0}$, $\mathbf{V}_0(\mathbf{q}) = \mathbf{0}$ and $\mathbf{B}_0(\mathbf{q}) = \mathbf{0}$. We modified the PX4 firmware [82] to expose the normalized thrust and torque being sent to the motors. The firmware's normalization of thrust and torque is unknown, and in fact, is learned from data via the input gain matrix $\mathbf{B}(\mathbf{q})$. We collected 12 state-control trajectories by flying the quadrotor from a starting pose to 12 different poses using a PID controller provided by the PX4 flight controller [82]. The trajectories were used to generate a dataset $\mathcal{D} = \{t_{0:N}^{(i)}, \mathbf{q}_{0:N}^{(i)}, \dot{\mathbf{q}}_{0:N}^{(i)}, \mathbf{u}^{(i)}\}_{i=1}^D$ with $N = 1$ and $D = 10000$. We trained our model as described in Section V-D for 5000 steps.

The trained model was used with the control policy in Section VII-C to track different trajectories: a vertical circle, a vertical lemniscate, and a 3-D piecewise-linear trajectories. Figs. 12 and 13 show that we achieve better tracking performance using our learned dynamics model and energy-based control compared to the nominal model and the geometric controller in [79]. The tracking errors of our controller with a learned model improve by 2 – 4 times compared to those of geometric control based on the nominal model, as shown in Table II.

2) *Learning Quadrotor Dynamics With Extra Payload:* In this section, we demonstrate that after our dynamics model is trained, if there is a change in the quadrotor dynamics, e.g., an extra payload is added, we are able to update the dynamics quickly starting from the previously trained model. We attached a coffee can to the quadrotor frame [see Fig. 11(c)] to change the mass and inertia matrix of the robot. We, then, collected a new dataset by driving the quadrotor to 12 different poses, and trained our dynamics model for only 100 steps, initialized with the trained model in Section VII-E1.

TABLE II
POSITION TRACKING ERRORS USING NOMINAL AND LEARNED QUADROTOR MODELS WITH AND WITHOUT PAYLOAD

Model	Train with payload	Test with payload	Circle	Lemniscate	Piecewise-linear
Nominal	-	No	0.26 (m)	0.52 (m)	0.62 (m)
Learned	No	No	0.13(m)	0.14(m)	0.22(m)
Learned	No	Yes	0.20 (m)	0.40 (m)	0.30 (m)
Learned	Yes	Yes	0.13 (m)	0.12 (m)	0.21 (m)

The bold values are to emphasize that our method outperforms other baselines, i.e. achieving smaller errors.

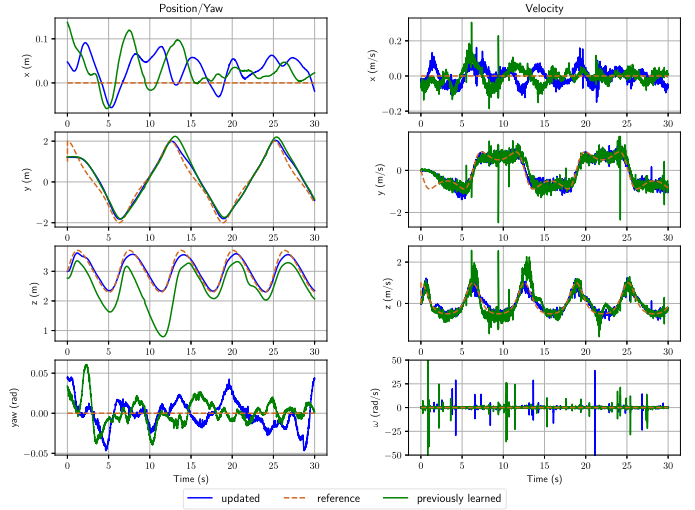


Fig. 15. Tracking a piecewise-linear trajectory with extra payload using our previously learned and updated models.

In the presence of the coffee can payload, the tracking performance of the controller with the previously learned model degrades as shown in Figs. 14 and 15. Meanwhile, after a quick model update, the robot is able to track desired trajectories accurately again. Table II shows that our updated model improves the tracking errors by 1.5–4 times compared to the previously learned model.

VIII. CONCLUSION

This article proposed a neural ODE network design for robot dynamics learning that captures Lie group kinematics, e.g., SE(3), and port-Hamiltonian dynamics constraints by construction. It also developed a general control approach for trajectory tracking based on the learned Lie group port-Hamiltonian dynamics. The learning and control designs were not system specific, and thus, can be applied to different types of robots whose states evolve on Lie group. These techniques had the potential to enable robots to quickly adapt their models online, in response to changing operational conditions or structural damage, and continue to maintain stability during autonomous operation. Future work will focus on extending our formulation to allow learning multirigid-body dynamics, handling contact, and online adaptation to disturbances and structural changes in the dynamics.

APPENDIX

A. Implementation Details

We used fully connected neural networks whose architecture is shown as follows. The first number is the input dimension, while the last number is the output dimension. The numbers in between are the hidden layers' dimensions and activation functions. The value of ε_v and ε_ω in (40) is set to 0.01.

- 1) Pendulum
 - a) Input dimension: 9. Action dimension: 1.
 - b) $\mathbf{L}_v(\mathbf{q})$: 9 - 300 Tanh - 300 Tanh - 300 Tanh - 300 Linear - 6.
 - c) $\mathcal{V}_\theta(\mathbf{q})$: 9 - 50 Tanh - 50 Tanh - 50 Linear - 1.
 - d) $\mathbf{B}_\theta(\mathbf{q})$: 9 - 300 Tanh - 300 Tanh - 300 Linear - 3.
- 2) Pybullet quadrotor
 - a) Input dimension: 12. Action dimension: 4.
 - b) $\mathbf{L}_v(\mathbf{q})$ only takes the position $\mathbf{p} \in \mathbb{R}^3$ as input: 3 - 400 Tanh - 400 Tanh - 400 Tanh - 400 Linear - 6.
 - c) $\mathbf{L}_\omega(\mathbf{q})$ only takes the rotation matrix $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ as input: 9 - 400 Tanh - 400 Tanh - 400 Tanh - 400 Linear - 6.
 - d) $\mathcal{V}_\theta(\mathbf{q})$: 12 - 400 Tanh - 400 Tanh - 400 Linear - 1.
 - e) $\mathbf{B}_\theta(\mathbf{q})$: 12 - 400 Tanh - 400 Tanh - 400 Linear - 24.
- 3) Real PX4 quadrotor
 - a) Input dimension: 12. Action dimension: 4.
 - b) $\mathbf{L}_v(\mathbf{q})$ only takes the position $\mathbf{p} \in \mathbb{R}^3$ as input: 3 - 20 Tanh - 20 Tanh - 20 Tanh - 20 Linear - 6.
 - c) $\mathbf{L}_\omega(\mathbf{q})$ only takes the rotation matrix $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ as input: 9 - 20 Tanh - 20 Tanh - 20 Tanh - 20 Linear - 6.
 - d) $\mathbf{D}_{v;\theta}(\mathbf{q})$ only takes the position $\mathbf{p} \in \mathbb{R}^3$ as input: 3 - 20 Tanh - 20 Tanh - 20 Tanh - 20 Linear - 6.
 - e) $\mathbf{D}_{\omega;\theta}(\mathbf{q})$ only takes the rotation matrix $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ as input: 9 - 20 Tanh - 20 Tanh - 20 Tanh - 20 Linear - 6.
 - f) $V(\mathbf{q})$: 12 - 20 Tanh - 20 Tanh - 20 Linear - 1.
 - g) $\mathbf{B}_\theta(\mathbf{q})$: 12 - 20 Tanh - 20 Tanh - 20 Linear - 24.

B. Derivation of Hamiltonian Dynamics on SE(3) From Hamiltonian Dynamics on a Matrix Lie Group

The Hamiltonian dynamics on SE(3) in (30) can be obtained from the general matrix Lie group Hamiltonian dynamics in (18) by computing explicit expressions for the terms $\text{ad}_\xi^*(\mathbf{p})$ and $\mathbf{T}_e^* \mathbf{L}_q(\boldsymbol{\eta})$ with $\boldsymbol{\eta} = \frac{\partial \mathcal{H}(\mathbf{q}, \mathbf{p})}{\partial \mathbf{q}}$.

To obtain an explicit expression for $\mathbf{T}_e^* \mathbf{L}_q(\boldsymbol{\eta})$, we use (6) and the pairing in Definition 1:

$$\begin{aligned} \langle \mathbf{T}_e^* \mathbf{L}_q(\boldsymbol{\eta}), \boldsymbol{\xi} \rangle &= \langle \boldsymbol{\eta}, \mathbf{T}_e \mathbf{L}_q(\boldsymbol{\xi}) \rangle = \langle \boldsymbol{\eta}, \mathbf{q} \boldsymbol{\xi} \rangle \\ &= \text{tr}(\boldsymbol{\eta}^\top \mathbf{q} \boldsymbol{\xi}) = \langle \mathbf{q}^\top \boldsymbol{\eta}, \boldsymbol{\xi} \rangle. \end{aligned} \quad (80)$$

Thus, $\mathbf{T}_e^* \mathbf{L}_q(\boldsymbol{\eta}) = \mathbf{P}_{\mathbf{g}^*}(\mathbf{q}^\top \boldsymbol{\eta})$, where $\mathbf{P}_{\mathbf{g}^*}$ is an orthogonal projector on $\mathbf{g}^*$ [83, Def. 3.60], which depends on the specific matrix Lie group. For example, on SE(3) [84] with $\mathbf{A} \in \mathbb{R}^{3 \times 3}$,

$\mathbf{a}, \mathbf{b} \in \mathbb{R}^3$, and $c \in \mathbb{R}$

$$\mathbf{P}_{\mathbf{g}^*} \left(\begin{bmatrix} \mathbf{A} & \mathbf{a} \\ \mathbf{b}^\top & c \end{bmatrix} \right) = \begin{bmatrix} \frac{1}{2}(\mathbf{A} - \mathbf{A}^\top) & \mathbf{a} \\ \mathbf{0}^\top & 0 \end{bmatrix}. \quad (81)$$

To obtain an explicit expression for $\text{ad}_\xi^*(\mathbf{p})$, we use Definition 9 and the pairing in Definition 1 as

$$\begin{aligned} \langle \text{ad}_\xi^*(\mathbf{p}), \boldsymbol{\psi} \rangle &= \langle \mathbf{p}, \text{ad}_\xi(\boldsymbol{\psi}) \rangle = \langle \mathbf{p}, [\boldsymbol{\xi}, \boldsymbol{\psi}] \rangle \\ &= \text{tr}(\mathbf{p}^\top (\boldsymbol{\xi} \boldsymbol{\psi} - \boldsymbol{\psi} \boldsymbol{\xi})) = \langle [\boldsymbol{\xi}^\top, \mathbf{p}], \boldsymbol{\psi} \rangle. \end{aligned} \quad (82)$$

Thus, $\text{ad}_\xi^*(\mathbf{p}) = \mathbf{P}_{\mathbf{g}^*}([\boldsymbol{\xi}^\top, \mathbf{p}])$.

1) Expression for $\mathbf{T}_e^* \mathbf{L}_q(\boldsymbol{\eta})$ on SE(3): On SE(3), we have

$$\mathbf{q} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}^\top & 1 \end{bmatrix}, \quad \boldsymbol{\xi} = \begin{bmatrix} \hat{\boldsymbol{\omega}} & \mathbf{v} \\ \mathbf{0}^\top & 0 \end{bmatrix}, \quad \boldsymbol{\eta} = \begin{bmatrix} \boldsymbol{\eta}_R & \boldsymbol{\eta}_p \\ \mathbf{0}^\top & 0 \end{bmatrix} \quad (83)$$

and

$$\begin{aligned} \langle \boldsymbol{\eta}, w \mathbf{q} \boldsymbol{\xi} \rangle &= \langle \boldsymbol{\eta}_R, \mathbf{R} \hat{\boldsymbol{\omega}} \rangle + \langle \boldsymbol{\eta}_p, \mathbf{R} \mathbf{v} \rangle \\ &= \left\langle \frac{1}{2}(\mathbf{R}^\top \boldsymbol{\eta}_R - \boldsymbol{\eta}_R^\top \mathbf{R}), \hat{\boldsymbol{\omega}} \right\rangle + \langle \mathbf{R}^\top \boldsymbol{\eta}_p, \mathbf{v} \rangle \\ &= \left\langle \begin{bmatrix} \frac{1}{2}(\mathbf{R}^\top \boldsymbol{\eta}_R - \boldsymbol{\eta}_R^\top \mathbf{R}) & \mathbf{R}^\top \boldsymbol{\eta}_p \\ \mathbf{0}^\top & 0 \end{bmatrix}, \begin{bmatrix} \hat{\boldsymbol{\omega}} & \mathbf{v} \\ \mathbf{0}^\top & 0 \end{bmatrix} \right\rangle \\ &= \langle \mathbf{T}_e^* \mathbf{L}_q(\boldsymbol{\eta}), \boldsymbol{\xi} \rangle \end{aligned} \quad (84)$$

where we used the properties $\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$ and $\text{tr}(\hat{\mathbf{x}}\mathbf{A}) = \frac{1}{2}\text{tr}(\hat{\mathbf{x}}(\mathbf{A} - \mathbf{A}^\top))$ of the hat map in the second equality.

2) Expression for $\text{ad}_\xi^*(\mathbf{p})$ on SE(3): On $\mathfrak{se}(3)$, we have

$$\mathbf{p} = \begin{bmatrix} \hat{\mathbf{a}} & \mathbf{b} \\ \mathbf{0}^\top & 0 \end{bmatrix}, \quad \boldsymbol{\xi} = \begin{bmatrix} \hat{\boldsymbol{\omega}} & \mathbf{v} \\ \mathbf{0}^\top & 0 \end{bmatrix}, \quad \boldsymbol{\psi} = \begin{bmatrix} \hat{\mathbf{c}} & \mathbf{d} \\ \mathbf{0}^\top & 0 \end{bmatrix} \quad (85)$$

and

$$\begin{aligned} \langle \mathbf{p}, [\boldsymbol{\xi}, \boldsymbol{\psi}] \rangle &= \left\langle \begin{bmatrix} \hat{\mathbf{a}} & \mathbf{b} \\ \mathbf{0}^\top & 0 \end{bmatrix}, \begin{bmatrix} [\hat{\boldsymbol{\omega}}, \hat{\mathbf{c}}] & \hat{\boldsymbol{\omega}} \mathbf{d} + \hat{\mathbf{v}} \mathbf{c} \\ \mathbf{0} & 0 \end{bmatrix} \right\rangle \\ &= \langle \hat{\mathbf{a}}, [\hat{\boldsymbol{\omega}}, \hat{\mathbf{c}}] \rangle + \langle \mathbf{b}, \hat{\boldsymbol{\omega}} \mathbf{d} + \hat{\mathbf{v}} \mathbf{c} \rangle \\ &= \langle [\hat{\mathbf{a}}, \hat{\boldsymbol{\omega}}], \hat{\mathbf{c}} \rangle + \langle \mathbf{b}, \hat{\mathbf{v}} \mathbf{c} \rangle + \langle \mathbf{b}, \hat{\boldsymbol{\omega}} \mathbf{d} \rangle \\ &= \langle [\hat{\mathbf{a}}, \hat{\boldsymbol{\omega}}], \hat{\mathbf{c}} \rangle - \text{tr}(\mathbf{v} \mathbf{b}^\top \hat{\mathbf{c}}) + \langle \hat{\boldsymbol{\omega}}^\top \mathbf{b}, \mathbf{d} \rangle \\ &= \langle [\hat{\mathbf{a}}, \hat{\boldsymbol{\omega}}], \hat{\mathbf{c}} \rangle + \left\langle \frac{1}{2}(\mathbf{v} \mathbf{b}^\top - \mathbf{b} \mathbf{v}^\top), \hat{\mathbf{c}} \right\rangle + \langle \hat{\mathbf{b}} \boldsymbol{\omega}, \mathbf{d} \rangle \\ &= \left\langle [\hat{\mathbf{a}}, \hat{\boldsymbol{\omega}}] + \frac{1}{2}[\hat{\mathbf{b}}, \hat{\mathbf{v}}], \hat{\mathbf{c}} \right\rangle + \langle \hat{\mathbf{b}} \boldsymbol{\omega}, \mathbf{d} \rangle \\ &= \left\langle \begin{bmatrix} [\hat{\mathbf{a}}, \hat{\boldsymbol{\omega}}] + \frac{1}{2}[\hat{\mathbf{b}}, \hat{\mathbf{v}}] & \hat{\mathbf{b}} \boldsymbol{\omega} \\ \mathbf{0}^\top & 0 \end{bmatrix}, \begin{bmatrix} \hat{\mathbf{c}} & \mathbf{d} \\ \mathbf{0}^\top & 0 \end{bmatrix} \right\rangle \\ &= \langle \text{ad}_\xi^*(\mathbf{p}), \boldsymbol{\psi} \rangle \end{aligned} \quad (86)$$

where we used the hat map properties $\hat{\mathbf{x}}^\top = -\hat{\mathbf{x}}$, $\hat{\mathbf{x}}\mathbf{y} = -\hat{\mathbf{y}}\mathbf{x}$, and $\text{tr}(\hat{\mathbf{x}}\mathbf{A}) = \frac{1}{2}\text{tr}(\hat{\mathbf{x}}(\mathbf{A} - \mathbf{A}^\top))$.

3) Consistency Between Hamiltonian Dynamics on a Matrix Lie Group and on SE(3): Denote the momentum in (17) as

$\mathbf{p}_\xi = \begin{bmatrix} \hat{\mathbf{p}}_\omega & \mathbf{p}_v \\ \mathbf{0}^\top & 0 \end{bmatrix}$ and the momentum in (26) as $\mathbf{p}_\zeta = \begin{bmatrix} \mathbf{p}_\omega \\ \mathbf{p}_v \end{bmatrix}$, where $\xi = \hat{\zeta}$. Let $\mathbf{p}_\omega = \frac{\partial \mathcal{L}}{\partial \omega} = \hat{\mu}$. By the chain rule, we have

$$\mathbf{p}_{\omega_i} = \frac{\partial \mathcal{L}}{\partial \omega_i} = \left\langle \hat{\mu}, \frac{\partial \hat{\omega}}{\partial \omega_i} \right\rangle = 2\mu_i \quad (87)$$

where $\omega = [\omega_1 \ \omega_2 \ \omega_3]^\top$, $\mu = [\mu_1 \ \mu_2 \ \mu_3]^\top$, or

$$\mathbf{p}_\omega = \frac{1}{2} \hat{\mathbf{p}}_\omega. \quad (88)$$

Therefore, we have

$$\mathbf{p}_\xi = \begin{bmatrix} \frac{1}{2} \hat{\mathbf{p}}_\omega & \mathbf{p}_v \\ \mathbf{0}^\top & 0 \end{bmatrix}, \text{ i.e., } \mathbf{a} = \frac{1}{2} \hat{\mathbf{p}}_\omega, \mathbf{b} = \mathbf{p}_v \quad (89)$$

leading to

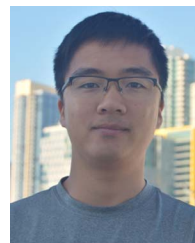
$$\text{ad}_\xi^*(\mathbf{p}_\xi) = \begin{bmatrix} \left(\frac{1}{2} \hat{\mathbf{p}}_\omega \omega + \frac{1}{2} \hat{\mathbf{p}}_v \mathbf{v} \right)^\wedge & \hat{\mathbf{p}}_v \omega \\ \mathbf{0}^\top & 0 \end{bmatrix} \quad (90)$$

By plugging in (89), (84), and (90) in the matrix Lie group Hamiltonian dynamics (18), we obtain the Hamiltonian dynamics on SE(3) in (30).

REFERENCES

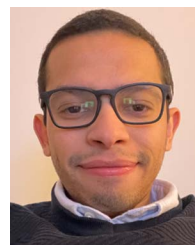
- [1] L. Ljung, "System identification," in *Wiley Encyclopedia of Electrical and Electronics Engineering*. Hoboken, NJ, USA: Wiley, 1999, pp. 1–19.
- [2] D. Nguyen-Tuong and J. Peters, "Model learning for robot control: A survey," *Cogn. Process.*, vol. 12, pp. 319–340, 2011.
- [3] M. Deisenroth and C. Rasmussen, "PILCO: A model-based and data-efficient approach to policy search," in *Proc. Int. Conf. Mach. Learn.*, 2011, pp. 465–472.
- [4] G. Williams et al., "Information theoretic MPC for model-based reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2017, pp. 1714–1721.
- [5] M. Raissi, P. Perdikaris, and G. Karniadakis, "Multistep neural networks for data-driven discovery of nonlinear dynamical systems," 2018, *arXiv:1801.01236*.
- [6] K. Chua, R. Calandra, R. McAllister, and S. Levine, "Deep reinforcement learning in a handful of trials using probabilistic dynamics models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 4759–4770.
- [7] M. Lutter and J. Peters, "Combining physics and deep learning to learn continuous-time dynamics models," *Int. J. Robot. Res.*, vol. 42, pp. 83–107, 2023.
- [8] J. Gupta, K. Menda, Z. Manchester, and M. Kochenderfer, "A general framework for structured learning of mechanical systems," 2019, *arXiv:1902.08705*.
- [9] M. Cranmer, S. Greydanus, S. Hoyer, P. Battaglia, D. Spergel, and S. Ho, "Lagrangian neural networks," in *Proc. ICLR Workshop Integration Deep Neural Models Differ. Equ.*, 2020.
- [10] S. Greydanus, M. Dzamba, and J. Yosinski, "Hamiltonian neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 15379–15389.
- [11] Z. Chen, J. Zhang, M. Arjovsky, and L. Bottou, "Symplectic recurrent neural networks," in *Proc. Int. Conf. Learn. Representations*, 2020.
- [12] M. Roehrl, T. Runkler, V. Brandstetter, M. Tokic, and S. Obermayer, "Modeling system dynamics with physics-informed neural networks based on Lagrangian mechanics," *IFAC-PapersOnLine*, vol. 53, pp. 9195–9200, 2020.
- [13] Y. Lu, S. Lin, G. Chen, and J. Pan, "ModLaNets: Learning generalisable dynamics via modularity and physical inductive bias," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 14384–14397.
- [14] C. Neary and U. Topcu, "Compositional learning of dynamical system models using port-Hamiltonian neural networks," in *Proc. Learn. Dyn. Control Conf.*, 2023, pp. 679–691.
- [15] A. Sanchez-Gonzalez et al., "Graph networks as learnable physics engines for inference and control," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4470–4479.
- [16] M. Lutter, C. Ritter, and J. Peters, "Deep lagrangian networks: Using physics as model prior for deep learning," in *Proc. Int. Conf. Learn. Representations*, 2018.
- [17] B. Hall, *Lie Groups, Lie Algebras, and Representations*. New York, NY, USA: Springer, 2013.
- [18] K. Lynch and F. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge, U.K.: Cambridge Univ. Press, 2017.
- [19] L. Falorsi and P. Forré, "Neural ordinary differential equations on manifolds," 2020, *arXiv:2006.06663*.
- [20] K. Elamvazhuthi, X. Zhang, S. Oymak, and F. Pasqualetti, "Learning on manifolds: Universal approximations properties using geometric controllability conditions for neural ODEs," in *Proc. Learn. Dyn. Control Conf.*, 2023, pp. 1–11.
- [21] A. Lou et al., "Neural manifold ordinary differential equations," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 17548–17558.
- [22] Y. Wotte, F. Califano, and S. Stramigioli, "Optimal potential shaping on SE(3) via neural ordinary differential equations on Lie groups," *Int. J. Robot. Res.*, 2024, doi: [10.1177/02783649241256044](https://doi.org/10.1177/02783649241256044).
- [23] A. I. Lurie, *Analytical Mechanics*. Berlin, Germany: Springer, 2013.
- [24] D. Holm, *Geometric Mechanics*. Singapore: World Scientific, 2008.
- [25] T. Bertalan, F. Dietrich, I. Mezić, and I. Kevrekidis, "On learning Hamiltonian systems from data," *Chaos, Interdiscipl. J. Nonlinear Sci.*, vol. 12, 2019, Art. no. 121107.
- [26] M. Finzi, K. A. Wang, and A. Wilson, "Simplifying Hamiltonian and Lagrangian neural networks via explicit constraints," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 13880–13889.
- [27] Y. D. Zhong, B. Dey, and A. Chakraborty, "Symplectic ODE-Net: Learning Hamiltonian dynamics with control," in *Proc. Int. Conf. Learn. Representations*, 2019.
- [28] J. Willard, X. Jia, S. Xu, M. Steinbach, and V. Kumar, "Integrating Scientific Knowledge with Machine Learning for Engineering and Environmental Systems," *ACM Comput. Surv.*, vol. 55, no. 4, 2022, Art. no. 66, doi: [10.1145/3514228](https://doi.org/10.1145/3514228).
- [29] A. Maciejewski, "Hamiltonian formalism for euler parameters," *Celestial Mechanics*, vol. 37, pp. 47–57, 1985.
- [30] R. Shivarama and E. Fahrendthold, "Hamilton's equations with euler parameters for rigid body dynamics modeling," *J. Dyn. Syst., Meas., Control*, vol. 126, pp. 124–130, 2004.
- [31] R. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural ordinary differential equations," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 6572–6583.
- [32] T. Duong and N. Atanasov, "Hamiltonian-based neural ODE networks on the SE(3) manifold for dynamics learning and control," in *Proc. Robot., Sci. Syst.*, 2021.
- [33] T. Lee, M. Leok, and N. H. McClamroch, *Global Formulations of Lagrangian and Hamiltonian Dynamics on Manifolds*. Berlin, Germany: Springer, 2017.
- [34] Y. D. Zhong, B. Dey, and A. Chakraborty, "Dissipative SymODEN: Encoding Hamiltonian dynamics with dissipation and control into deep learning," in *Proc. ICLR Workshop Integration Deep Neural Models Differ. Equ.*, 2020.
- [35] Y. Song, A. Romero, M. Müller, V. Koltun, and D. Scaramuzza, "Reaching the limit in autonomous racing: Optimal control versus reinforcement learning," *Sci. Robot.*, vol. 8, 2023, Art. no. eadg1462.
- [36] T. Salzmann, E. Kaufmann, J. Arrizabalaga, M. Pavone, D. Scaramuzza, and M. Ryll, "Real-time neural MPC: Deep learning model predictive control for quadrotors and agile robotic platforms," *IEEE Robot. Automat. Lett.*, vol. 8, no. 4, pp. 2397–2404, Apr. 2023.
- [37] D. Scaramuzza and E. Kaufmann, "Learning agile, vision-based drone flight: From simulation to reality," in *Proc. Int. Symp. Robot. Res.*, 2022, pp. 11–18.
- [38] A. Loquercio, E. Kaufmann, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, "Learning high-speed flight in the wild," *Sci. Robot.*, vol. 6, 2021, Art. no. eabg5810.
- [39] J. Ibarz, J. Tan, C. Finn, M. Kalakrishnan, P. Pastor, and S. Levine, "How to train your robot with deep reinforcement learning: Lessons we have learned," *Int. J. Robot. Res.*, vol. 40, pp. 698–721, 2021.
- [40] T. X. Nghiem et al., "Physics-informed machine learning for modeling and control of dynamical systems," in *Proc. Amer. Control Conf.*, 2023, pp. 3735–3750.
- [41] F. Djeumou, C. Neary, E. Goubault, S. Putot, and U. Topcu, "Neural networks with physics-informed architectures and constraints for dynamical systems modeling," in *Proc. Learn. Dyn. Control Conf.*, 2022, pp. 263–277.
- [42] A. Thorpe, C. Neary, F. Djeumou, M. Oishi, and U. Topcu, "Physics-informed kernel embeddings: Integrating prior system knowledge with data-driven control," in *Proc. Amer. Control Conf.*, 2024.

- [43] L. Ruthotto and E. Haber, "Deep neural networks motivated by partial differential equations," *J. Math. Imag. Vis.*, vol. 62, pp. 352–364, 2019.
- [44] R. Wang, R. Walters, and R. Yu, "Incorporating symmetry into deep dynamics models for improved generalization," in *Proc. Int. Conf. Learn. Representations*, 2021.
- [45] M. Lutter, K. Listmann, and J. Peters, "Deep Lagrangian networks for end-to-end learning of energy-based control for under-actuated systems," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 7718–7725.
- [46] T. Beckers, J. Seidman, P. Perdikaris, and G. Pappas, "Gaussian process port-Hamiltonian systems: Bayesian learning with physics prior," in *Proc. IEEE Conf. Decis. Control*, 2022, pp. 1447–1453.
- [47] T. Beckers, "Data-driven Bayesian control of port-Hamiltonian systems," in *Proc. IEEE Conf. Decis. Control*, 2023, pp. 8708–8713.
- [48] B. Leimkuhler and S. Reich, *Simulating Hamiltonian Dynamics*. Cambridge, U.K.: Cambridge Univ. Press, 2004.
- [49] P. Toth, D. J. Rezende, A. Jaegle, S. Racanière, A. Botev, and I. Higgins, "Hamiltonian generative networks," in *Proc. Int. Conf. Learn. Representations*, 2019.
- [50] J. Mason, C. Allen-Blanchette, N. Zolman, E. Davison, and N. Leonard, "Learning interpretable dynamics from images of a freely rotating 3D rigid body," in *Proc. Third Symp. Knowl. guided Mach. Learn.*, 2022. [Online]. Available: <https://aaai.org/conference/fall-symposia/fss22/>.
- [51] A. Van Der Schaft and D. Jeltsema, "Port-Hamiltonian systems theory: An introductory overview," *Found. Trends Syst. Control*, vol. 1, pp. 173–378, 2014.
- [52] J. E. Marsden and M. West, "Discrete mechanics and variational integrators," *Acta Numerica*, vol. 10, pp. 357–514, 2001.
- [53] A. Havens and G. Chowdhary, "Forced variational integrator networks for prediction and control of mechanical systems," in *Proc. Learn. Dyn. Control*, 2021, pp. 1142–1153.
- [54] V. Duruisseaux, T. P. Duong, M. Leok, and N. Atanasov, "Lie group forced variational integrator networks for learning and control of robot systems," in *Proc. Learn. Dyn. Control Conf.*, 2023, pp. 731–744.
- [55] R. Chen and M. Tao, "Data-driven prediction of general Hamiltonian dynamics via learning exactly-symplectic maps," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 1717–1727.
- [56] O. So, G. Li, E. Theodorou, and M. Tao, "Data-driven discovery of non-newtonian astronomy via learning non-euclidean hamiltonian," in *Proc. ICML Mach. Learn. Phys. Sci. Workshop*, 2022.
- [57] F. Borrelli, A. Bemporad, and M. Morari, *Predictive Control for Linear and Hybrid Systems*. Cambridge, U.K.: Cambridge Univ. Press, 2017.
- [58] R. Ortega, M. Spong, F. Gómez-Estern, and G. Blankenstein, "Stabilization of a class of underactuated mechanical systems via interconnection and damping assignment," *IEEE Trans. Autom. Control*, vol. 47, no. 8, pp. 1218–1233, Aug. 2002.
- [59] J. Á. Acosta, M. Sanchez, and A. Ollero, "Robust control of underactuated aerial manipulators via IDA-PBC," in *Proc. IEEE Conf. Decis. Control*, 2014, pp. 673–678.
- [60] O. Cieza and J. Reger, "IDA-PBC for underactuated mechanical systems in implicit port-hamiltonian representation," in *Proc. Eur. Control Conf.*, 2019, pp. 614–619.
- [61] Z. Wang and P. Goldsmith, "Modified energy-balancing-based control for the tracking problem," *IET Control Theory Appl.*, vol. 2, pp. 310–322, 2008.
- [62] C. Souza, V. Raffo, and E. Castelan, "Passivity-based control of a quadrotor UAV," *IFAC Proc. Vols.*, vol. 47, pp. 3196–3201, 2014.
- [63] L. Furieri, C. L. Galimberti, M. Zakwan, and G. Ferrari-Trecate, "Distributed neural network control with dependability guarantees: A compositional port-Hamiltonian approach," in *Proc. Learn. Dyn. Control Conf.*, 2022, pp. 571–583.
- [64] C. L. Galimberti, L. Furieri, L. Xu, and G. Ferrari-Trecate, "Hamiltonian deep neural networks guaranteeing nonvanishing gradients by design," *IEEE Trans. Autom. Control*, vol. 68, no. 5, pp. 3155–3162, May 2023.
- [65] E. Sebastián, T. Duong, N. Atanasov, E. Montijano, and C. Sagüés, "LEMURS: Learning distributed multi-robot interactions," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2023, pp. 7713–7719.
- [66] J. Marsden and T. Ratiu, *Introduction to Mechanics and Symmetry: A Basic Exposition of Classical Mechanical Systems*. Berlin, Germany: Springer, 2013.
- [67] Y. Zhou, C. Barnes, J. Lu, J. Yang, and H. Li, "On the continuity of rotation representations in neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 5745–5753.
- [68] T. Barfoot, *State Estimation for Robotics*. Cambridge, U.K.: Cambridge Univ. Press, 2017.
- [69] E. G. Hemingway and O. M. O'Reilly, "Perspectives on euler angle singularities, gimbal lock, and the orthogonality of applied forces and applied moments," *Multibody Syst. Dyn.*, vol. 44, pp. 31–56, 2018.
- [70] M. Faessler, A. Franchi, and D. Scaramuzza, "Differential flatness of quadrotor dynamics subject to rotor drag for accurate tracking of high-speed trajectories," *IEEE Robot. Automat. Lett.*, vol. 3, no. 2, pp. 620–626, Apr. 2018.
- [71] P. Forni, D. Jeltsema, and G. Lopes, "Port-Hamiltonian formulation of rigid-body attitude control," *IFAC-PapersOnLine*, vol. 48, pp. 164–169, 2015.
- [72] R. Rashad, F. Califano, and S. Stramigioli, "Port-hamiltonian passivity-based control on SE(3) of a fully actuated UAV for aerial physical interaction near-hovering," *IEEE Robot. Automat. Lett.*, vol. 4, no. 4, pp. 4378–4385, Oct. 2019.
- [73] J. Delmerico and D. Scaramuzza, "A benchmark comparison of monocular visual-inertial odometry algorithms for flying robots," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2018, pp. 2502–2509.
- [74] S. A. S. Mohamed, M. Haghbayan, T. Westerlund, J. Heikkonen, H. Tenhunen, and J. Plosila, "A survey on odometry for autonomous navigation systems," *IEEE Access*, vol. 7, pp. 97466–97486, 2019.
- [75] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, vol. 32, pp. 8026–8037.
- [76] J. R. Dormand and P. J. Prince, "A family of embedded Runge-Kutta formulae," *J. Comput. Appl. Math.*, vol. 6, pp. 19–26, 1980.
- [77] M. Lienen and S. Günnemann, "torchode: A parallel ODE solver for PyTorch," in *Proc. Symbiosis Deep Learn. Differ. Equ. II*, 2022.
- [78] A. Iserles, H. Z. Munthe-Kaas, S. P. Nørsett, and A. Zanna, "Lie-group methods," *Acta Numerica*, vol. 9, pp. 215–365, 2000.
- [79] T. Lee, M. Leok, and N. H. McClamroch, "Geometric tracking control of a quadrotor UAV on SE(3)," in *Proc. IEEE Conf. Decis. Control*, 2010, pp. 5420–5425.
- [80] H. K. Khalil, *Nonlinear Systems*. Upper Saddle River, NJ, USA: Prentice-Hall, 2002.
- [81] J. Panerati, H. Zheng, S. Zhou, J. Xu, A. Prorok, and A. Schöllig, "Learning to fly: A PyBullet gym environment to learn the control of multiple nano-quadcopters," 2020. [Online]. Available: <https://github.com/utiasDSL/gym-pybullet-drones>
- [82] L. Meier, D. Honegger, and M. Pollefeys, "PX4: A node-based multi-threaded open source robotics framework for deeply embedded platforms," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2015, pp. 6235–6240.
- [83] N. Boumal, *An Introduction to Optimization on Smooth Manifolds*. Cambridge, U.K.: Cambridge Univ. Press, 2023. [Online]. Available: <https://www.nicolasboumal.net/book>
- [84] P. Arathoon, "Coadjoint orbits of the special Euclidean group," Master's thesis, Dept. Math., Univ. of Manchester, Manchester, U.K., 2015.



Thai Duong (Graduate Student Member, IEEE) received the B.S. degree in electronics and telecommunications from the Hanoi University of Science and Technology, Hanoi, Vietnam, in 2011, and the M.S. degree in electrical and computer engineering from Oregon State University, Corvallis, OR, USA, in 2013. He is currently working toward the Ph.D. degree in electrical and computer engineering with the University of California San Diego, La Jolla, CA, USA.

His research interests include machine learning with applications to robotics, mapping, and active exploration using mobile robots, robot dynamics learning, planning, and control.



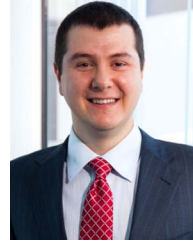
Abdullah Altawaitan (Student Member, IEEE) received the B.S. and M.S. degrees in electrical engineering from Arizona State University, Tempe, AZ, USA, in 2017 and 2019, respectively. He is currently working toward the Ph.D. degree in electrical and computer engineering with the University of California San Diego, La Jolla, CA, USA.

He is affiliated with the Electrical Engineering Department, College of Engineering and Petroleum, Kuwait University, Safat, Kuwait. His research interests include machine learning, control theory, and their applications to robotics.



Jason Stanley (Student Member, IEEE) received the B.S. degree in computer engineering in 2024 from the University of California San Diego (UCSD), La Jolla, CA, USA, where he is currently working toward the master's degree with the Department of Electrical and Computer Engineering.

His research interests include machine learning for robotics, control theory, and mobile robots.



Nikolay Atanasov (Senior Member, IEEE) received the B.S. degree in electrical engineering from Trinity College, Hartford, CT, USA, in 2008, and the M.S. and Ph.D. degrees in electrical and systems engineering from the University of Pennsylvania, Philadelphia, PA, USA, in 2012 and 2015, respectively.

He is an Associate Professor of electrical and computer engineering with the University of California San Diego, La Jolla, CA, USA. He works on probabilistic models that unify geometric and semantic information in simultaneous localization and map-

ping (SLAM) and on optimal control and reinforcement learning algorithms for minimizing probabilistic model uncertainty. His research interests include robotics, control theory, and machine learning, applied to active perception problems for autonomous mobile robots.

Dr. Atanasov was the recipient of the Joseph and Rosaline Wolf award for the best Ph.D. dissertation in electrical and systems engineering at the University of Pennsylvania in 2015, the Best Conference Paper Award at the IEEE International Conference on Robotics and Automation (ICRA) in 2017, the NSF CAREER Award in 2021, and the IEEE RAS Early Academic Career Award in Robotics and Automation in 2023.